



Instituto Superior de Engenharia de Lisboa

Departamento de Engenharia de Eletrónica e Telecomunicações e de Computadores
DEETC

Python com Spyder:
Instalação e Fundamentos



Vítor Manuel de Oliveira Fialho

Fevereiro de 2026

Índice

1	Introdução.....	1
2	Instalação do Software.....	3
2.1	Instalação do <i>Python</i>	3
2.2	Instalação do <i>Spyder</i>	5
3	Ambiente de desenvolvimento - <i>Spyder</i>.....	9
3.1	Descrição das Funcionalidades do IDE.....	10
3.2	Configurações Iniciais do IDE.....	13
3.3	Execução do script ‘ <i>Python_first_code.ipynb</i> ’.....	15
3.3.1	Execução em modo <i>debug</i>	16
3.3.2	Método Alternativo.....	18
3.4	Integração da Placa de Som em <i>Python</i>	18
4	Introdução à Linguagem <i>Python</i>.....	21
4.1	Tipos de Variáveis.....	22
4.1.1	Conversão de Tipos de Variáveis.....	23
4.2	Operações Básicas.....	24
4.2.1	Operações Aritméticas Simples.....	24
4.2.2	Operações Lógicas.....	25
4.2.3	Operações Aritméticas com Arrays.....	27
4.2.4	Instruções Especiais com <i>Arrays</i>	29
4.2.5	Instruções Especiais com Matrizes (<i>Arrays</i> multi-dimensionais).....	31
4.3	Controlo de Fluxo.....	34
4.3.1	Decisão Binária <i>if-else</i>	34
4.3.2	Ciclo <i>for</i>	35
4.3.3	Ciclo <i>while</i>	35
4.3.4	<i>Match-Case</i>	36
4.4	Representação Gráfica.....	37
4.5	Funções.....	39
4.6	Bibliotecas e Funções de Uso Corrente.....	45
4.6.1	<i>NumPy</i>	45
4.6.2	<i>Matplotlib</i> (<i>matplotlib.pyplot</i>).....	46
4.6.3	<i>SciPy</i>	46
4.6.4	<i>NumPy</i> FFT (<i>np.fft</i>).....	47
4.7	Links Úteis.....	47

Lista de Figuras

Figura 2.1: Instalação do <i>Python</i> (Passo 1).....	3
Figura 2.2: Instalação do <i>Python</i> (Passo 2).....	4
Figura 2.3: Instalação do <i>Python</i> (Passo 3).....	4
Figura 2.4: Instalação do <i>Python</i> (Passo 4).....	4
Figura 2.5: Instalação do <i>Spyder</i> (Passo 1 – site oficial).....	5
Figura 2.6: <i>Windows Defender</i>	5
Figura 2.7: Instalação do <i>Spyder</i> (Passo 2, 3 e 4).....	6
Figura 2.8: Instalação do <i>Spyder</i> (Passo 5 e 6).....	6
Figura 2.9: Instalação do <i>Spyder</i> (Passo 7 e 8).....	7
Figura 2.10: Ambiente gráfico do <i>Spyder</i> após a primeira inicialização.....	7
Figura 3.1: Ambiente gráfico do <i>Spyder</i> (descrição do ambiente gráfico).....	10
Figura 3.2: Ambiente gráfico do <i>Spyder</i> (editor de texto).....	10
Figura 3.3: Ambiente gráfico do <i>Spyder</i> (consola).....	11
Figura 3.4: Ambiente gráfico do <i>Spyder</i> (<i>pane</i> multi-funcionalidade).....	11
Figura 3.5: Barra de tarefas com as múltiplas funcionalidades (Run file).....	12
Figura 3.6: Preparação do ambiente de execução (passo 1).....	13
Figura 3.7: Preparação do ambiente de execução (passo 2).....	14
Figura 3.8: Preparação do ambiente de execução (passo 3).....	14
Figura 3.9: Execução do primeiro <i>script Python</i> no IDE <i>Spyder</i>	15
Figura 3.10: Lista de variáveis após a execução do <i>script: Python_first_code.py</i>	16
Figura 3.11: Consola com os resultados após a execução do <i>script: Python_first_code.py</i>	16
Figura 3.12: Aspeto visual do menu de <i>debug</i> realizado no <i>script: Python_first_code.py</i>	17
Figura 3.13: Aspeto visual do processo de <i>debug</i> realizado no <i>script: Python_first_code.py</i>	17
Figura 3.14: Aspeto visual do separador das variáveis durante o processo de <i>debug</i>	17
Figura 3.15: Instruções em modo <i>ipy</i>	18
Figura 3.16: Mensagens na consola após a instalação da biblioteca <i>sounddevice</i> via <i>conda</i> ...19	
Figura 3.17: <i>Spyder</i> IDE após execução do <i>script Introducao.py</i>	19
Figura 3.18: Resultado dos três gráficos do <i>script Introducao.py</i>	20
Figura 4.1: Representação gráfica do sinal sinusoidal.....	38
Figura 4.2: Representação dos sinais resultantes do exemplo apresentado na tabela 4.42.....	42
Figura 4.3: Resultado do código da tabela 4.44 - Sinal com ruído - tempo e histograma.....	44

Lista de Tabelas

Tabela 3.1: Significado dos ícones da barra de tarefas do <i>Spyder</i>	12
Tabela 4.1: Resumo das variáveis mais usadas em <i>Python</i>	22
Tabela 4.2: Conversão para inteiro.....	23
Tabela 4.3: Conversão para <i>float</i>	23
Tabela 4.4: Conversão para número complexo.....	23
Tabela 4.5: Verificação do tipo de variável.....	23
Tabela 4.6: Exemplo com variáveis int e float usadas em <i>Python</i>	24
Tabela 4.7: Resumo de algumas operações aritméticas simples usadas em <i>Python</i>	24
Tabela 4.8: Exemplo do operador resto de divisão inteira.....	24
Tabela 4.9: Operadores de Comparação.....	25
Tabela 4.10: Operadores Lógicos.....	25
Tabela 4.11: Operador XOR <i>bitwise</i>	26
Tabela 4.12: Operadores lógicos: exemplo com variáveis numéricas.....	26
Tabela 4.13: Operadores lógicos: exemplo aplicado a um conjunto de valores (<i>array</i>).....	26
Tabela 4.14: Criação de um vetor uni dimensional (<i>array</i>).....	27
Tabela 4.15: Aceder ao conteúdo de um <i>array</i>	27
Tabela 4.16: Aceder ao conteúdo de um <i>array</i>	27
Tabela 4.17: Exemplo de soma entre dois <i>arrays</i>	28
Tabela 4.18: Exemplo de subtração entre dois <i>arrays</i>	28
Tabela 4.19: Exemplo de multiplicação entre dois <i>arrays</i>	28
Tabela 4.20: Exemplo de potenciação de um <i>array</i> com multiplicação por uma constante.....	28
Tabela 4.21: Seleção de Elementos (Indexação Lógica): função <i>arange()</i>	29
Tabela 4.22: Seleção de Elementos (Indexação Lógica): função <i>linspace()</i>	29
Tabela 4.23: Modificação de Elementos.....	30
Tabela 4.24: Repetição de Valores.....	30
Tabela 4.25: Concatenação de <i>Arrays</i>	30
Tabela 4.26: Inversão Temporal (espelhamento).....	30
Tabela 4.27: Exemplos de funções de cálculo com <i>arrays</i>	31
Tabela 4.28: Exemplos de criação de listas.....	31
Tabela 4.29: Exemplo de conversão de lista para <i>array</i>	32
Tabela 4.30: Criação de um <i>array</i> multi-dimensional (matriz).....	32
Tabela 4.31: Exemplos de operações com <i>arrays</i> multi-dimensionais.....	32
Tabela 4.32: Exemplos com operações de cálculo matricial.....	33
Tabela 4.33: Exemplos de indexação com matrizes.....	33
Tabela 4.34: Exemplos de cálculo de energia, potência média e valor médio.....	34

Tabela 4.35: Exemplos de decisão binária.....	35
Tabela 4.36: Exemplos de um ciclo for.....	35
Tabela 4.37: Exemplos de um ciclo while.....	36
Tabela 4.38: Exemplos de um ciclo match-case.....	36
Tabela 4.39: Exemplos da criação de uma figura com um sinal sinusoidal.....	38
Tabela 4.40: Cálculo da energia, potência e valor médio do sinal sinusoidal.....	39
Tabela 4.41: Exemplo da função <i>myParabola</i> definida no ficheiro <i>exemploFuncoes.py</i>	40
Tabela 4.42: Chamada às funções apresentadas na tabela 4.41.....	41
Tabela 4.43: Função para cálculo da média e desvio padrão de um <i>array</i>	42
Tabela 4.44: Exemplo de representação de dois gráficos distintos na mesma figura (subplot).....	43

1 Introdução

O presente documento tem como principal objetivo apoiar a instalação do Python e do ambiente de desenvolvimento *Spyder* (*Integrated Development Environment* – IDE), adotado em algumas Unidades Curriculares tais como Sinais e Sistemas (LEIRT), Telecomunicações I (LEETC/LEIRT) e Comunicação Digital (LEIC).

O *Python* é uma linguagem de programação multi-paradigma, de alto nível, que suporta os paradigmas imperativo, orientado a objetos e funcional. Devido à sua sintaxe simples, elevada legibilidade e vasta disponibilidade de bibliotecas, tornou-se uma das linguagens mais utilizadas no ensino da programação e na investigação científica.

No contexto da Engenharia, o *Python* assume particular relevância por ser uma ferramenta gratuita e de código aberto (*Open Source*), combinando facilidade de utilização com elevado desempenho computacional. Através de bibliotecas especializadas, como NumPy, SciPy e Matplotlib, é possível realizar cálculo numérico, processamento de sinais e representação gráfica de resultados de forma eficiente e estruturada [1,2,3]. Para além disso, permite a comunicação e controlo de hardware, como Arduino ou Raspberry Pi, alargando o seu campo de aplicação.

O IDE *Spyder* (*Scientific Python Development Environment*) apresenta uma interface gráfica semelhante à do MATLAB/OCTAVE, facilitando a transição para estudantes já familiarizados com esses ambientes. Este ambiente integrado inclui editor de texto, consola interativa, explorador de variáveis e ferramentas de depuração (*debug*), proporcionando um conjunto de funcionalidades adequadas ao desenvolvimento e análise de aplicações científicas. As principais características deste software são apresentadas no Capítulo 3.

No Capítulo 4 apresentam-se diversos exemplos práticos de programação em *Python*, acompanhados das respectivas descrições teóricas e do correspondente código devidamente comentado, com especial enfoque na sua aplicação à análise e processamento de sinais [1,2,3,4].

Neste documento assume-se que os estudantes possuem conhecimentos prévios de programação, nomeadamente ao nível de declaração de variáveis, expressões condicionais, controlo de fluxo e definição de funções. O *Python* é aqui utilizado como ferramenta de apoio à análise e simulação dos conceitos abordados nas unidades curriculares, não constituindo, por si só, o objeto principal de estudo.

2 Instalação do Software

Neste capítulo descreve-se de forma sucinta a forma de instalação do *Python* e *Spyder* para o sistema operativo Windows. Recomenda-se que mantenham os softwares atualizados, pois são alvo de constantes atualizações.

2.1 Instalação do *Python*

Para instalar o *Python* deverá aceder ao site oficial e seleccionar a opção *Download*, conforme apresentado na figura 2.1

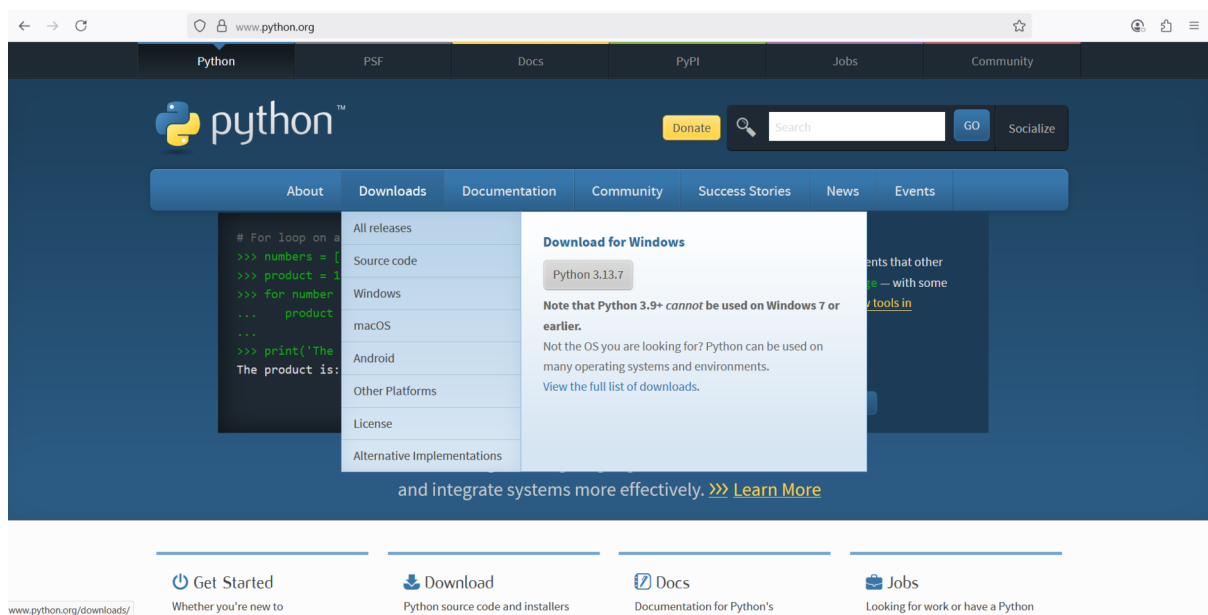


Figura 2.1: Instalação do *Python* (Passo 1)

Após executar o ficheiro *Spyder-Windows-x86_64.exe* surge esta janela de configuração, devendo manter a diretoria indicada no processo de instalação. Selecione as duas opções indicadas na figura 2.2.

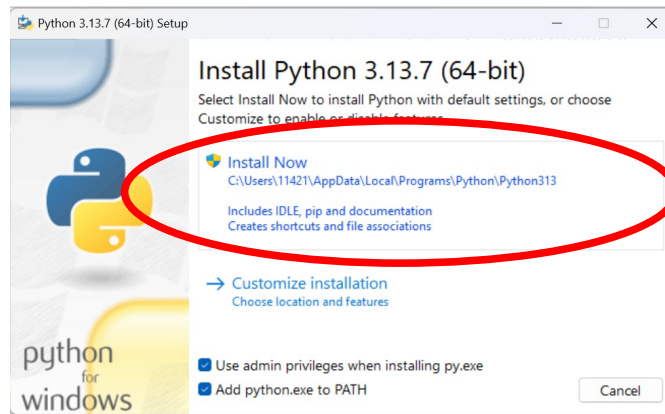


Figura 2.2: Instalação do *Python* (Passo 2)

Após o processo de instalação, será apresentada a janela indicada na figura 2.3, onde deverá selecionar a opção indicada. Assim que a instalação concluir com sucesso, será apresentada a janela da figura 2.4.

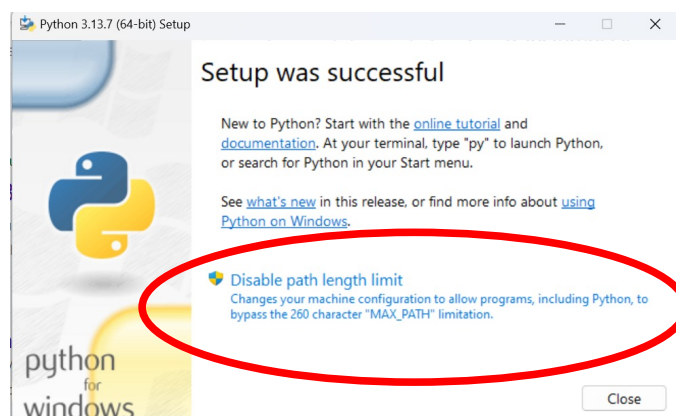


Figura 2.3: Instalação do *Python* (Passo 3)

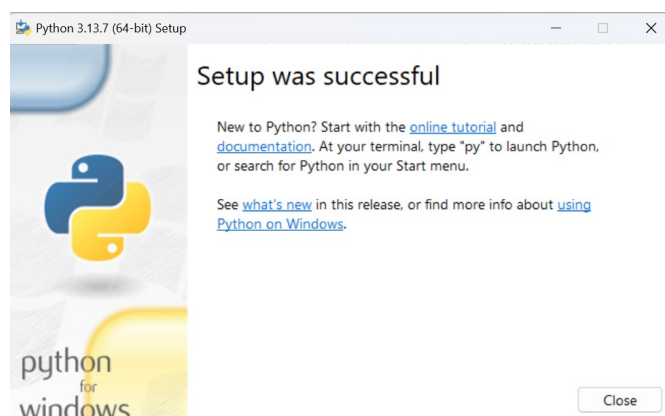


Figura 2.4: Instalação do *Python* (Passo 4)

2.2 Instalação do *Spyder*

Após a instalação do *Python*, deverá instalar o IDE *Spyder* conforme se descreve nesta secção. Sugere-se a instalação deste IDE já que possui diversas funcionalidades simples de usar e que facilitam a aprendizagem desta linguagem.

Aceda ao site oficial do *Spyder* (www.spyder-ide.org) e siga as instruções de instalação para o seu sistema operativo. Neste documento descrevem-se os passos para a instalação no Windows 10, conforme a apresentado na figura 2.5.

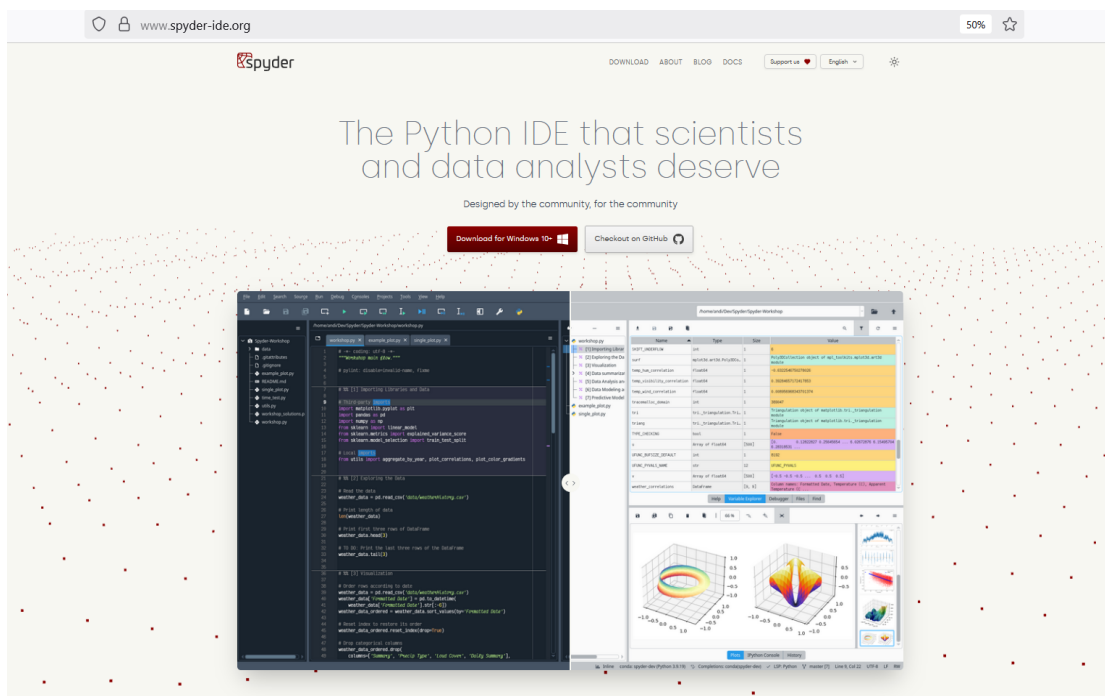


Figura 2.5: Instalação do *Spyder* (Passo 1 – site oficial)

Após seleccionar a opção que permite realizar o *download* poderá surgir a janela representada na figura 2.6, se assim for, aceite a instalação.



Figura 2.6: *Windows Defender*

A figura 2.7 apresenta o início do processo de instalação. Selecione a opção Seguinte (*Next*) e em seguida aceite os termos de licença de utilização do *Spyder*. Deverá instalar o *Python* na área do utilizador, conforme indicado.

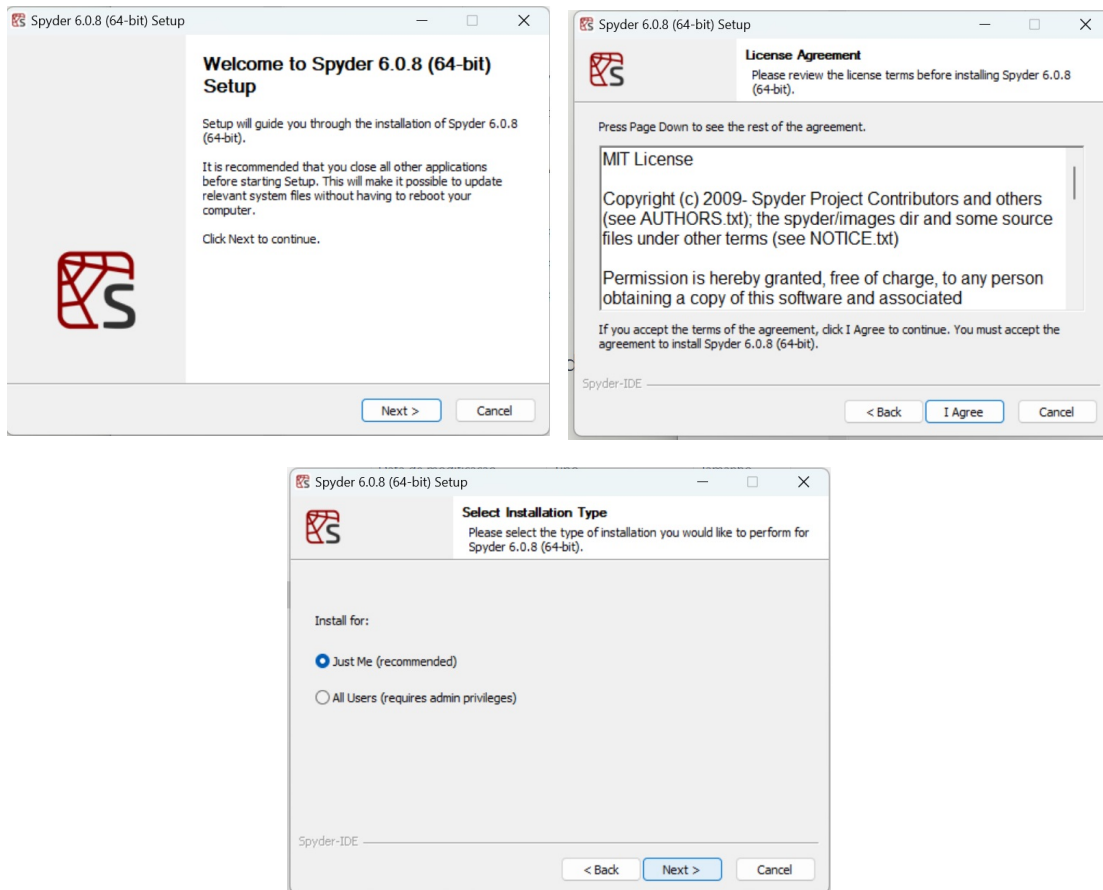


Figura 2.7: Instalação do *Spyder* (Passo 2, 3 e 4)

Após realizar os procedimentos anteriores, será apresentada a janela com a localização (*path*) onde será instalado o *Spyder*, conforme indicado na figura 2.8. Não se recomenda alterar o *path*. Recomenda-se que selecione as opções indicadas na janela da direita. Desta forma, não só terá acesso rápido ao *Spyder*, como todos os ficheiros temporários usados na instalação deste IDE serão removidos.

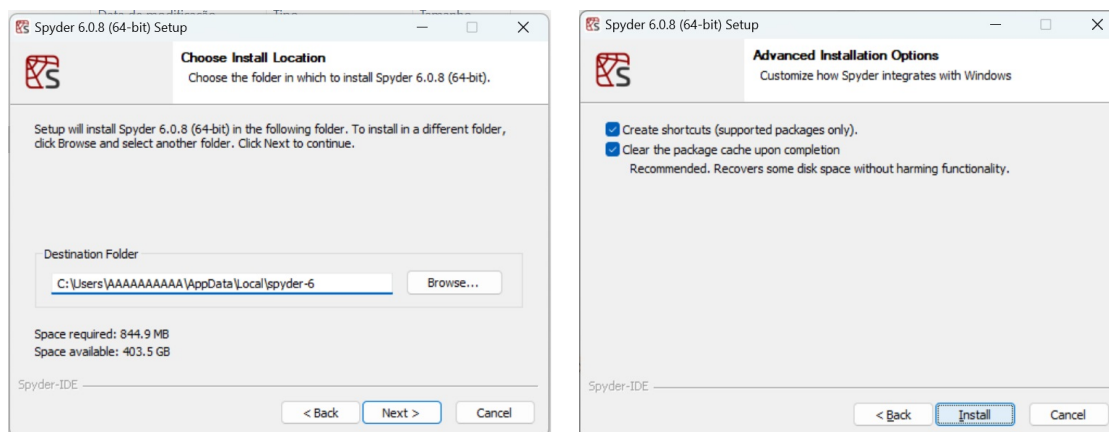


Figura 2.8: Instalação do *Spyder* (Passo 5 e 6)

Na figura 2.9 apresenta-se o processo de instalação finalizado. Assim que tal aconteça deverá carregar em Seguinte (*Next*) e em seguida aparecerá a janela com a indicação que a instalação está completa.

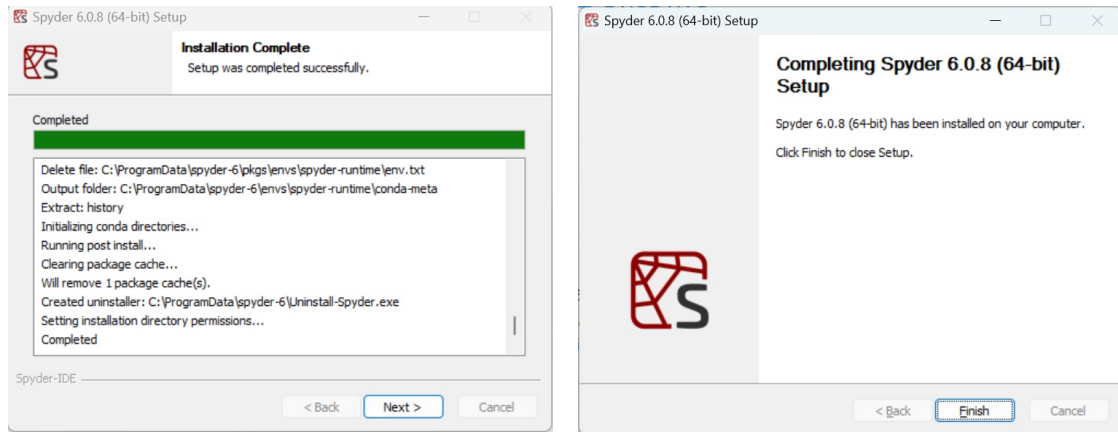


Figura 2.9: Instalação do *Spyder* (Passo 7 e 8)

A figura 2.10 apresenta o ambiente gráfico do IDE *Spyder*, este será descrito com mais detalhe no Capítulo 3.

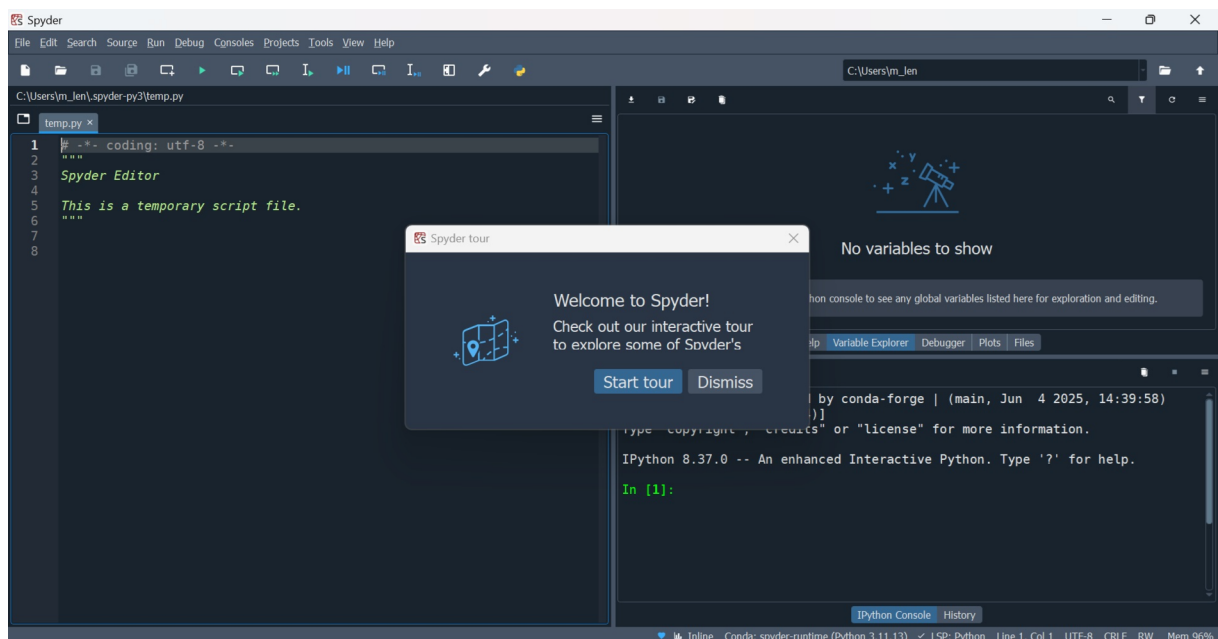


Figura 2.10: Ambiente gráfico do *Spyder* após a primeira inicialização

3 Ambiente de desenvolvimento - *Spyder*

Neste capítulo descrevem-se algumas funcionalidades do *Spyder* IDE, no entanto será apenas uma breve apresentação deste software de desenvolvimento. Para que o estudante tenha a capacidade de usar com a devida flexibilidade, terá de praticar através da sua utilização, como qualquer outra linguagem de programação.

Este IDE apresenta um ambiente gráfico muito semelhante ao MATLAB/OCTAVE, o que para quem já utilizou este software, a migração torna-se relativamente simples, comparativamente a outras linguagens de alto nível. A interface do *Spyder* é intuitiva pois apresenta um painel para visualização de variáveis, apresenta gráficos de forma imediata e apresenta um *debugger* integrado, o qual permite com relativa facilidade realizar *debug* ao software em desenvolvimento.

Na secção 3.1 descrevem-se as funcionalidades do IDE, onde são descritas as três principais áreas de trabalho do *Spyder* IDE. Na secção 3.2 apresenta-se a forma de testar um *script* ‘simples’ o qual contém *strings*, *arrays*, expressões de controlo de teste e controlo de fluxo. Na secção 3.3.1 o método para realizar *debug* no *Spyder*. Na secção 3.4 apresenta-se o *script* usado na aula teórica para iniciação ao *Python* no contexto da UC de Telecomunicações 1. Neste *script* são usados comandos especiais necessários para usar bibliotecas que permitem aceder à placa de som e desenhar gráficos. Neste exemplo em concreto será apresentada uma forma para desenhar um sinal amostrado e o respetivo histograma. Finalmente na secção 4.7 apresentam-se um conjunto de links úteis que o estudantes devem consultar para consolidar os conhecimentos em *Python*.

3.1 Descrição das Funcionalidades do IDE

Assim que se executa o *Spyder*, o ambiente gráfico é idêntico ao representado na figura 3.1. Conforme se verifica, este está dividido em três áreas distintas : editor de texto consola e uma terceira área de trabalho que permite aceder a múltiplas funcionalidades que se passam a descrever em seguida.

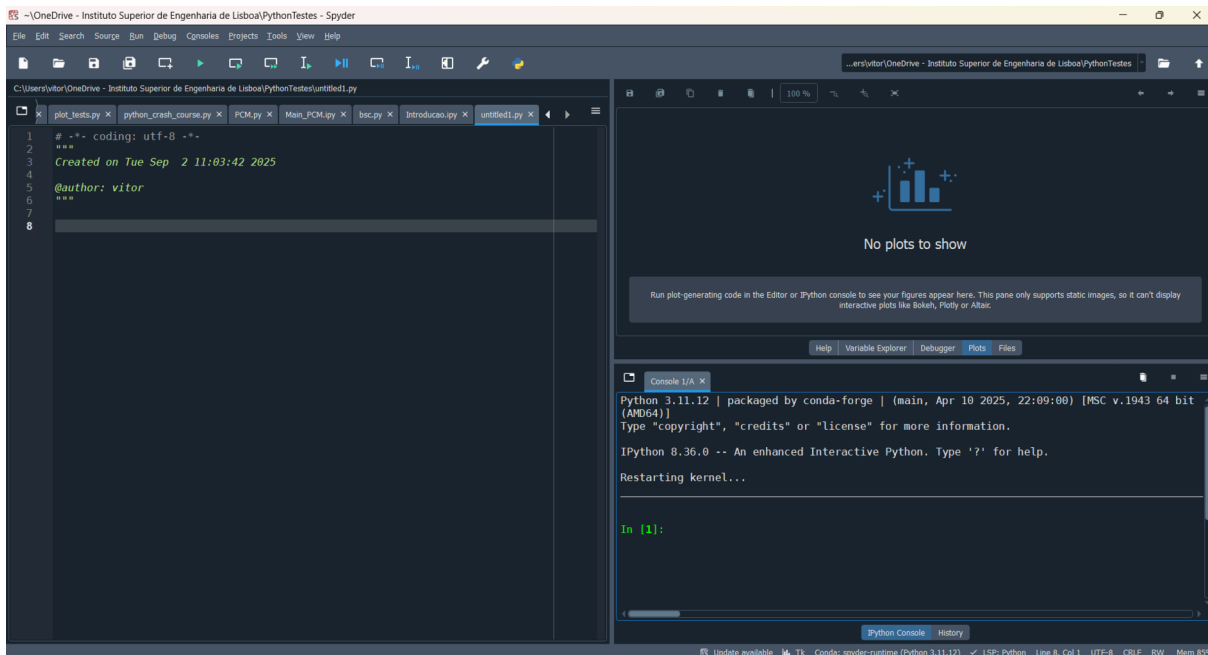


Figura 3.1: Ambiente gráfico do *Spyder* (descrição do ambiente gráfico)

Na figura 3.2 apresenta-se o editor de texto do IDE. Será neste editor onde o código *Python* será escrito e posteriormente testado. Os ficheiros deverão ter a extensão *.py* ou *ipy*.

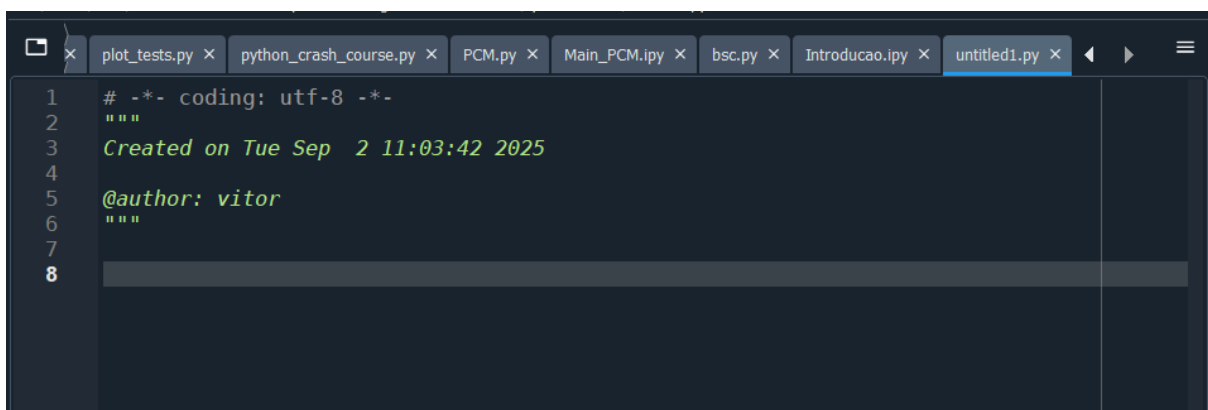


Figura 3.2: Ambiente gráfico do *Spyder* (editor de texto)

Na figura 3.3, apresenta-se a consola do *Spyder*, nesta serão apresentados os resultados (*print*) de variáveis, mensagens de erro e, visualização/interação em modo *debug*. É possível consultar o histórico dos comandos usado na opção *History*.

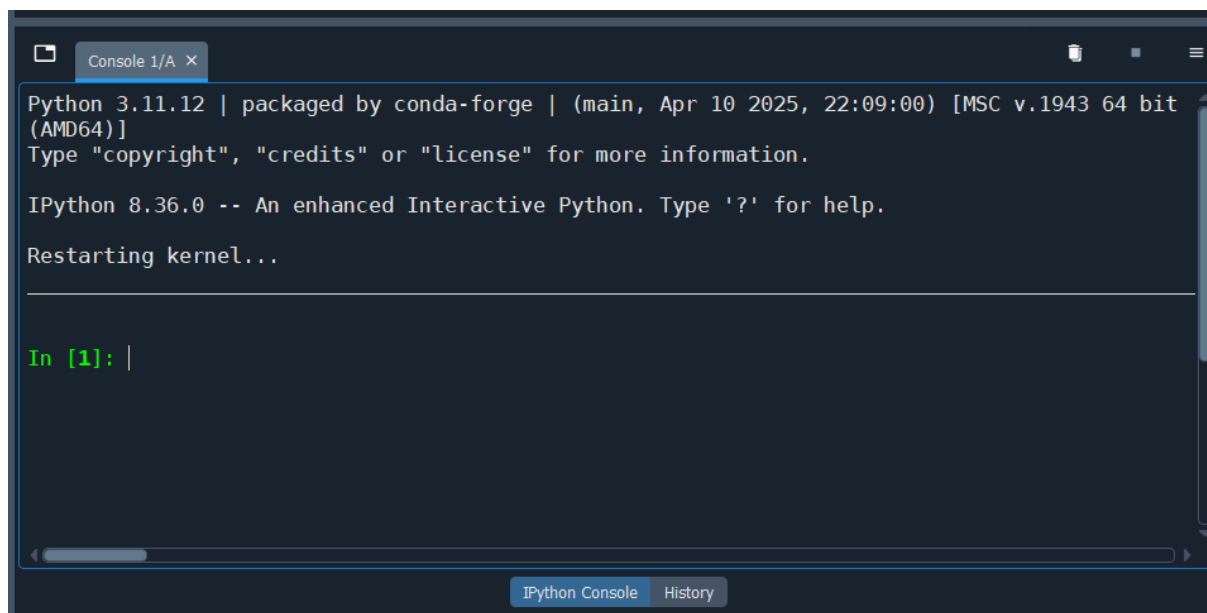


Figura 3.3: Ambiente gráfico do *Spyder* (consola)

Para além do editor de texto e consola, o *Spyder* apresenta uma terceira janela, a qual se apresenta na figura 3.4. Esta permite aceder a múltiplas funcionalidades, entre as quais: menus de ajuda, listagem de variáveis, *debugger*, figuras e ficheiros da diretoria de trabalho. Recomenda-se que esteja sempre seleccionada a opção *Variable Explorer*, pois assim será possível verificar o tipo de cada variável usada bem como o seu valor/conteúdo.

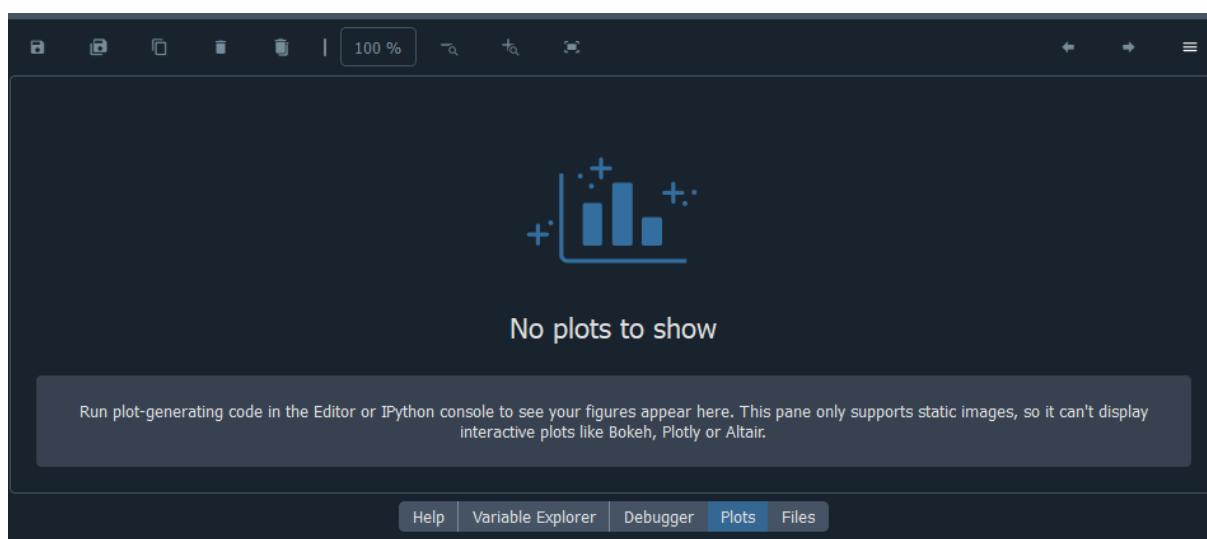


Figura 3.4: Ambiente gráfico do *Spyder* (*pane* multi-funcionalidade)

A barra de tarefas do *Spyder* está representada na figura 3.5. Como se verifica por cima figura está um número, o qual permite identificar a funcionalidade de cada ícone, conforme descrito na tabela 3.1.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

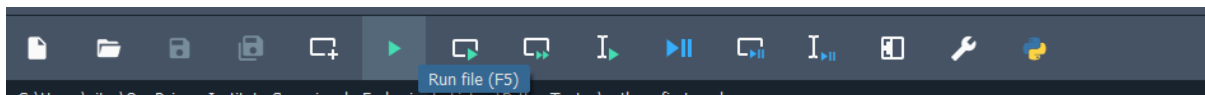


Figura 3.5: Barra de tarefas com as múltiplas funcionalidades (Run file)

As funcionalidade de 1 a 4 são de típica manipulação de ficheiros. A funcionalidade 5 introduz uma nova célula no código através da sequência de caracteres `#%%`. Esta funcionalidade pode ser útil caso se pretenda correr troços do código em vez do código completo.

A funcionalidade 6 permite executar o código na sua totalidade. Também é possível uma a tecla F5 como atalho. As funcionalidade 7 a 9 permitem executar apenas uma célula (caso exista) e a funcionalidade 9 possibilita correr apenas uma linha de código (que esteja seleccionada).

Todas as opções de *debug* são realizadas com recurso às funcionalidades 10 a 12, conforme se descreve com mais algum detalhe na secção 3.3.1. A funcionalidade 13 permite ocultar (ou não) a consola e a janela com o conjunto de variáveis. Recomenda-se que mantenha esta opção ativa, para assim verificar o estado das variáveis usadas e as mensagens apresentadas na consola. Os ícones 14 e 15 correspondem a opções de configuração do *Spyder* e *Python*, respetivamente. Nesta fase inicial recomenda-se que não se altere nenhuma das configurações por omissão.

Tabela 3.1: Significado dos ícones da barra de tarefas do *Spyder*

	Funcionalidade
1	Criar ficheiro
2	Abrir ficheiro
3 e 4	Gravar ficheiro(s)
5	Nova célula de código
6	Executar o <i>script</i>
7 e 8	Executar uma célula
9	Executar apenas uma linha de código
10 a 12	Funcionalidades do <i>debug</i>
13	Ocultar/Mostrar consola e lista de variáveis
14 e 15	Opções do <i>Spyder</i> e <i>Python</i>

3.2 Configurações Iniciais do IDE

Na fase de configuração do *Spyder*, é necessário definir duas opções relevantes para a execução de *scripts*: a eliminação das variáveis em memória e a configuração para que as imagens geradas sejam apresentadas em janelas de figuras independentes do IDE.

Para aceder ao menu que permite configurar a eliminação das variáveis em memória proceda à seguinte sequência, o que irá resultar no aparecimento da janela representada na figura 3.6 assinalada a vermelho.

Tools → *Preferences*

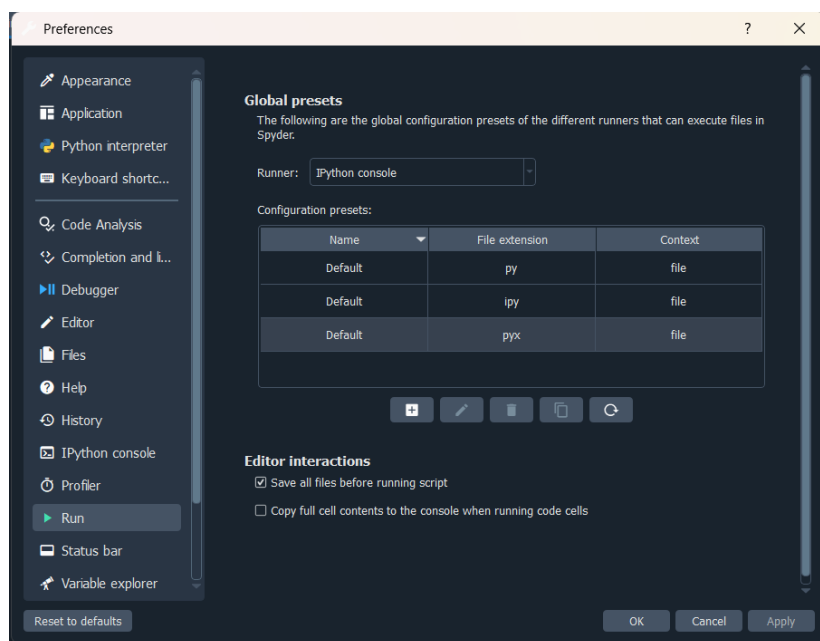


Figura 3.6: Preparação do ambiente de execução (passo 1)

Nesta janela, selecione a opção **Run**, conforme ilustrado na figura 3.6. Após essa seleção, serão apresentadas as opções assinaladas a verde. Selecione uma das linhas da tabela e em seguida selecione o lápis, conforme indicado na figura 3.7. Assim que seleccionar essa opção aparecerá a janela representada na figura 3.8 e selecione as opções indicadas nessa figura. Repita este procedimento para as restantes linhas da tabela. Assim que o fizer carregue no **Apply**. Aparecerá novamente a janela da figura 3.6 e carregue novamente no **Apply**.

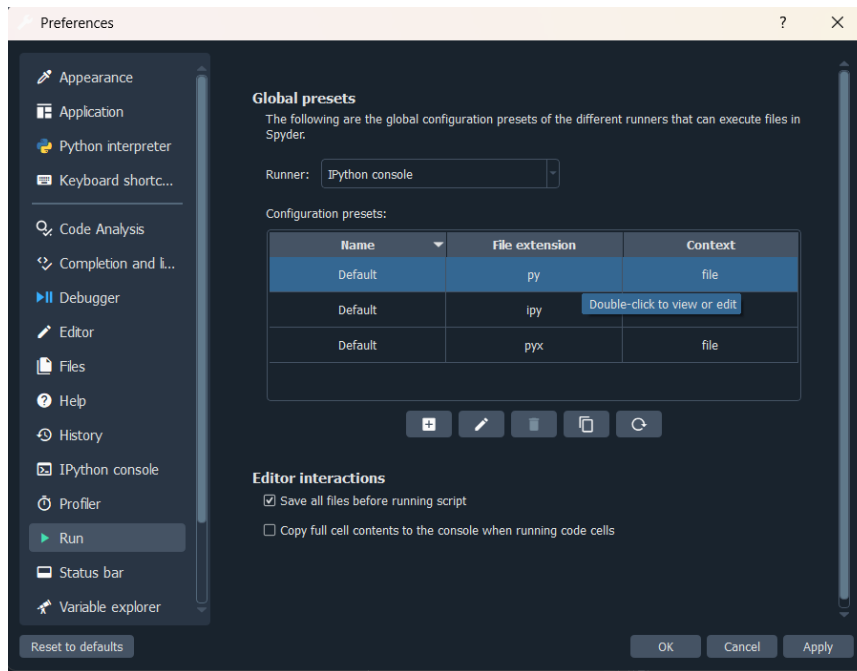


Figura 3.7: Preparação do ambiente de execução (passo 2)

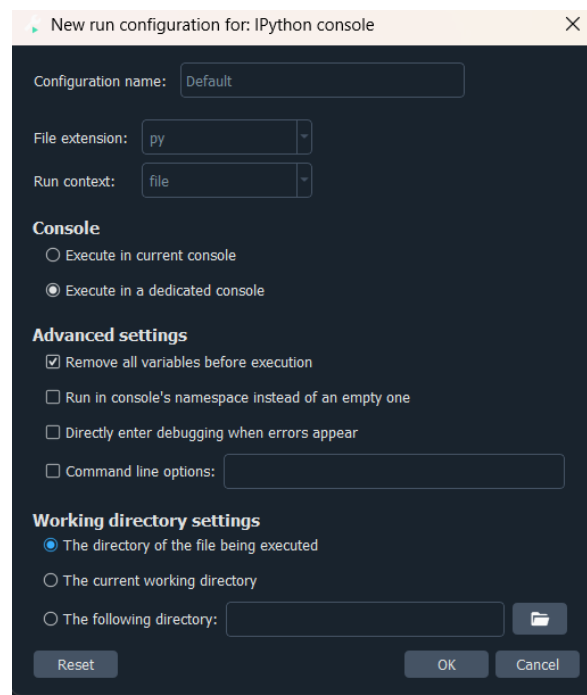


Figura 3.8: Preparação do ambiente de execução (passo 3)

3.3 Execução do script ‘Python_first_code.ipy’

Nesta secção descreve-se a forma como executar um *script* em *Python* através do *Spyder*. Apresenta-se de forma sucinta o uso de *strings*, números, variáveis *booleanas*, expressões de controlo de fluxo e o uso de uma função que retorna o resultado da soma entre dois algarismos.

Na figura 3.9 encontra-se apresentado no editor de texto o *script* *Python_first_code.py* disponibilizado na plataforma MOODLE. Para o obter aceda a plataforma e faça *download* para uma diretoria à sua escolha. Para abrir carregue em Abrir (ícone 2) e selecione o ficheiro. Para executar o *script* use o ícone 6 (ou tecla F5). Como resultado aparecerão as variáveis em memória conforme representado na figura 3.10. Há variáveis que são escritas na consola, conforme se apresenta na figura 3.11.

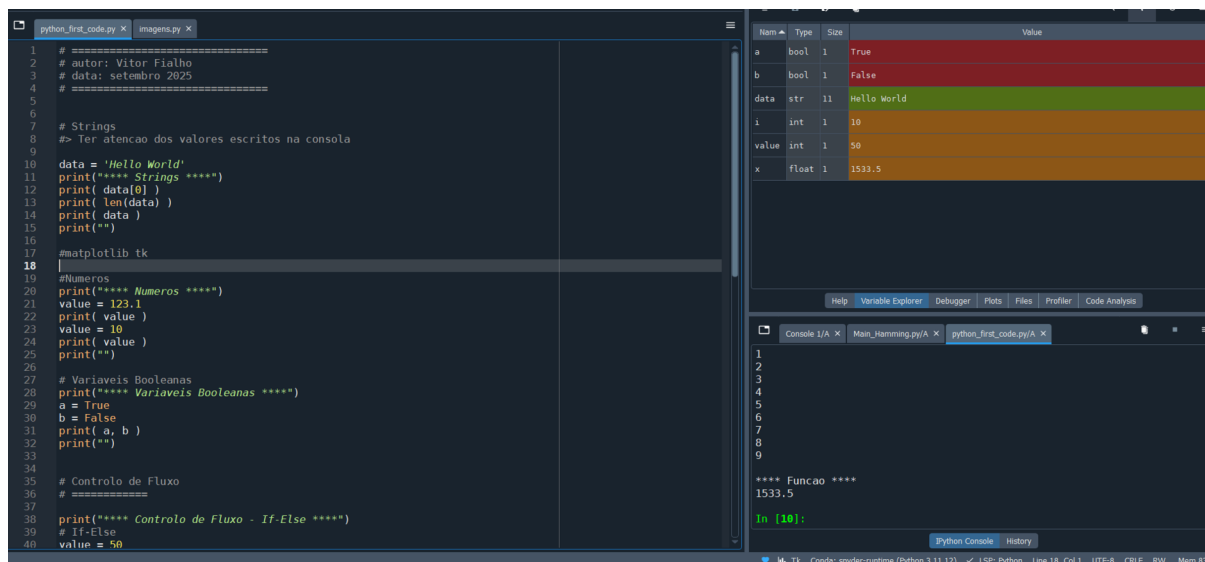
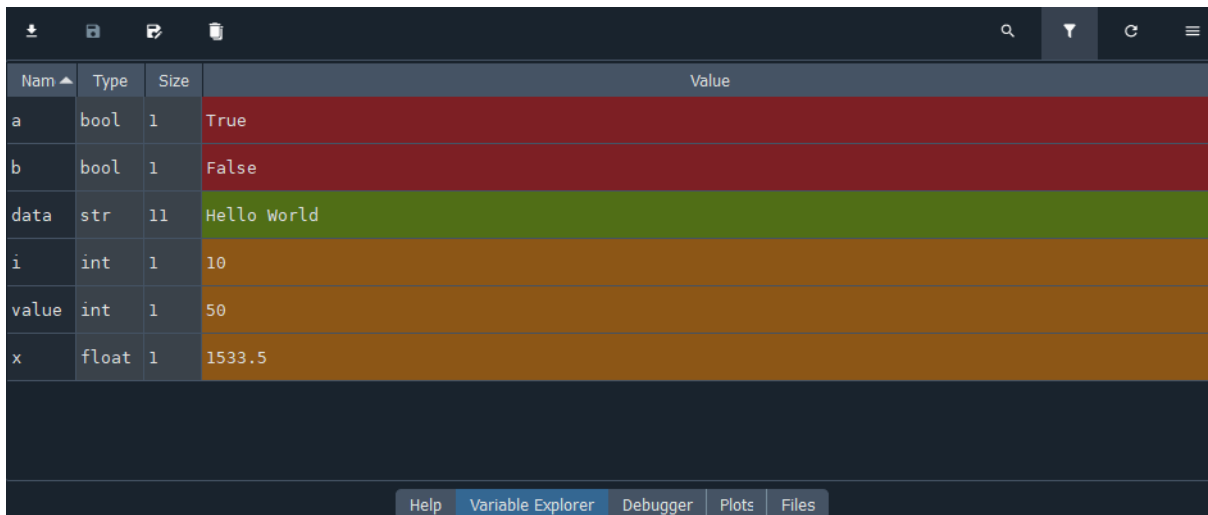
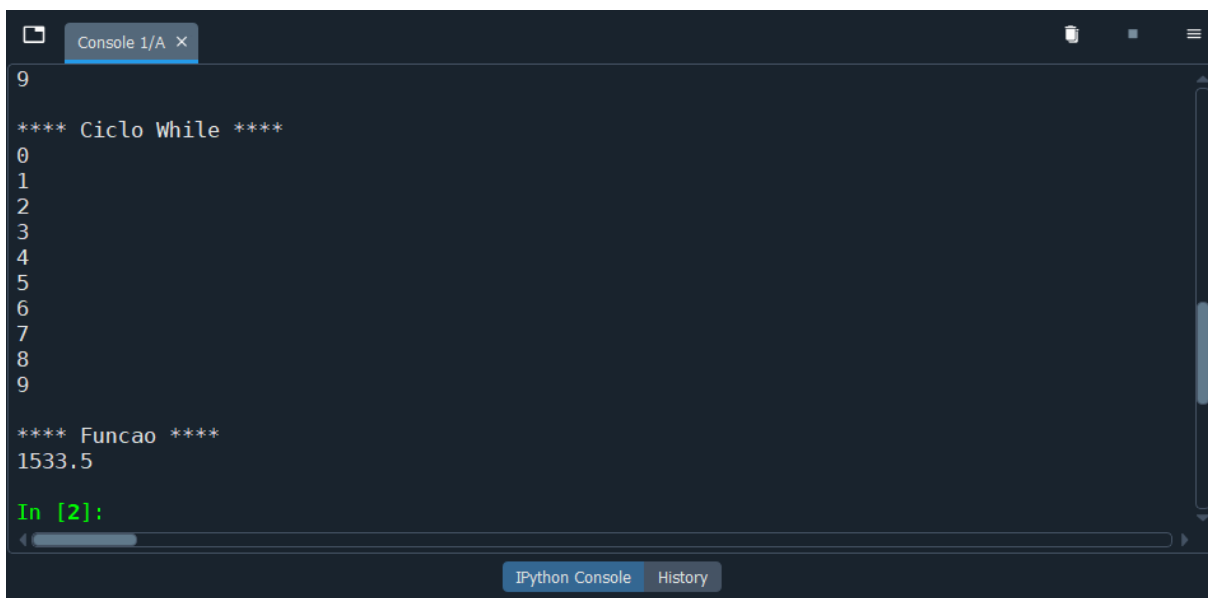


Figura 3.9: Execução do primeiro *script* *Python* no IDE *Spyder*



Nam	Type	Size	Value
a	bool	1	True
b	bool	1	False
data	str	11	Hello World
i	int	1	10
value	int	1	50
x	float	1	1533.5

Figura 3.10: Lista de variáveis após a execução do *script: Python_first_code.py*



```
9
**** Ciclo While ****
0
1
2
3
4
5
6
7
8
9

**** Funcao ****
1533.5

In [2]:
```

Figura 3.11: Consola com os resultados após a execução do *script: Python_first_code.py*

3.3.1 Execução em modo *debug*

Por vezes é necessário realizar *debug* ao software por forma a analisar e posterior deteção de erros. Para tal deverá colocar o cursor na linha onde pretende iniciar o processo, aceder ao menu *Debug* e seleccionar a opção *Set/Clear breakpoint* (tecla F12), conforme apresentado na figura 3.12. Após realizar este processo, aparecerá uma bola vermelha na linha de código seleccionada, conforme indicado na figura 3.13.

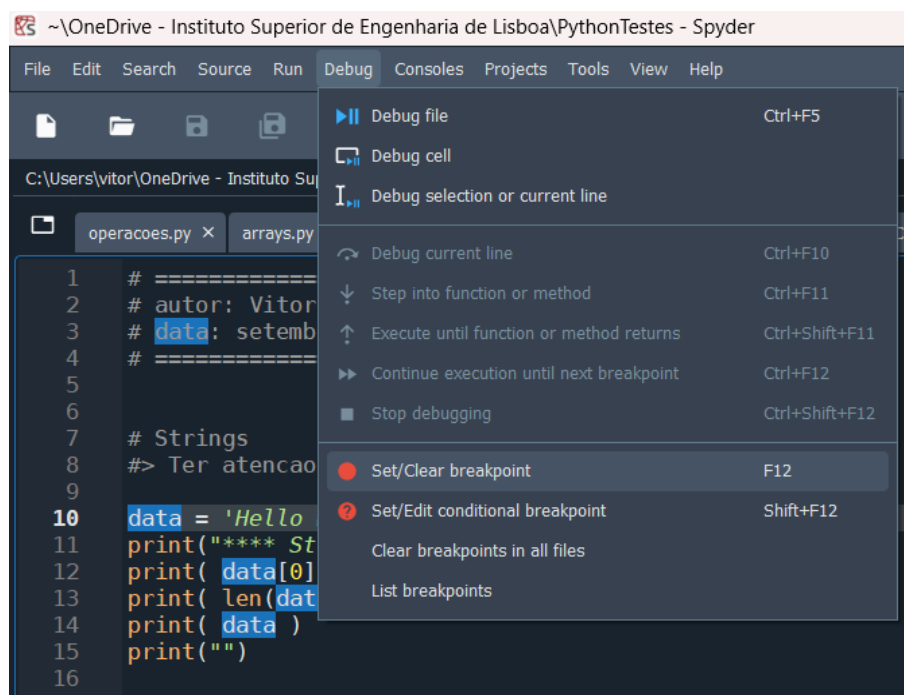


Figura 3.12: Aspetto visual do menu de *debug* realizado no script: *Python_first_code.py*

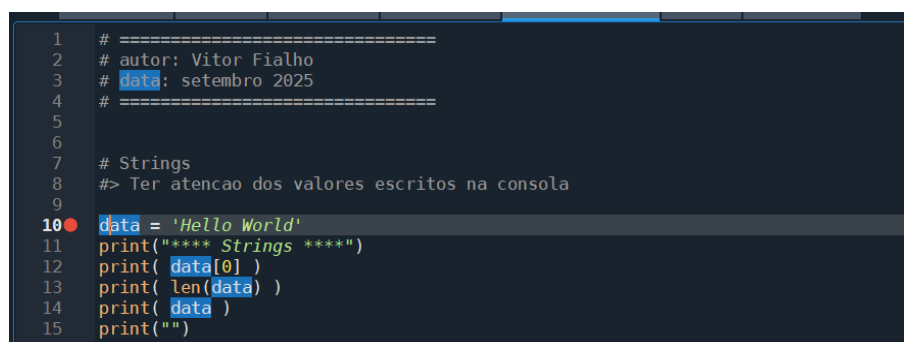


Figura 3.13: Aspetto visual do processo de *debug* realizado no script:
Python_first_code.py

Na figura 3.14 está representado o aspeto do separador onde estão indicados os valores das variáveis durante o processo de *debug*.

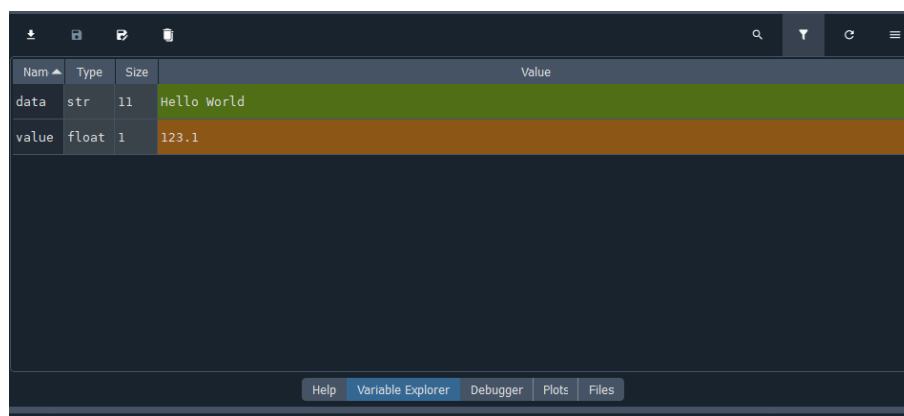
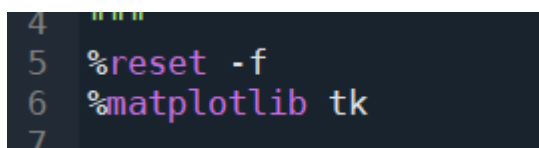


Figura 3.14: Aspetto visual do separador das variáveis durante o processo de *debug*

3.3.2 Método Alternativo

Caso não pretenda alterar as configurações por omissão do *Spyder* e realizar as ações descritas na secção 3.3.1 deverá usar as instruções indicadas na figura 3.15. Para tal deverá gravar o ficheiro com a extensão ‘ipy’.

A instrução da linha 5 `%reset -f` elimina as variáveis em memória existentes. A instrução da linha 6 `%matplotlib tk` recorre à biblioteca gráfica *Tkinter* (Tk) para representar os gráficos em janelas independentes do IDE. A opção de representar os gráficos em figuras independentes permite aceder a um menu mais completo de formatação e manipulação dos gráficos gerados.



```
4 %%
5 %reset -f
6 %matplotlib tk
7
```

Figura 3.15: Instruções em modo *ipy*

NOTA 1: por omissão o *Spyder* grava os ficheiros com extensão `.py`, no entanto, deverá gravar os ficheiros com extensão `.ipy`

A extensão *ipy* permite usar o *Spyder* no modo *Interactive Python*, o que permite utilizar tirar partido de comandos especiais (que usam como prefixo `%` ou `!`), permitindo assim criar figuras onde se pode realizar *zoom*, inspecionar as coordenadas, gravar com extensões distintas (png, jpeg...).

Em resumo:

`.py` → é a extensão padrão de um arquivo Python

`.ipy` → é uma extensão usada em arquivos do *IPython* (*Interactive Python*)

NOTA 2: caso seja necessário usar a funcionalidade de *debug*, descrita na secção 3.3.1, deverá alterar a extensão do ficheiro para `.py`

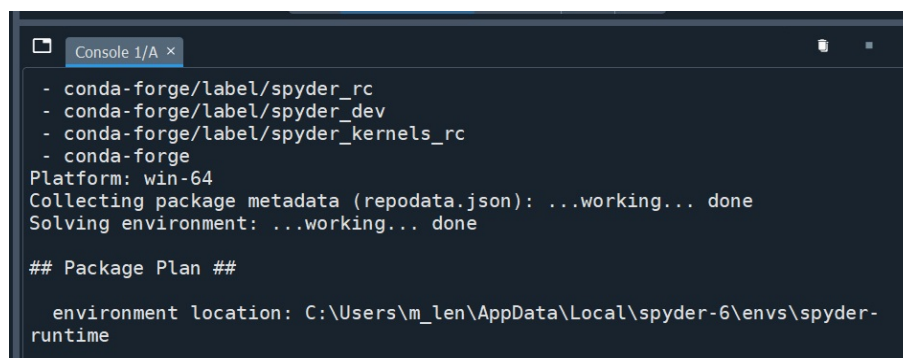
Recomenda-se, assim, que numa fase inicial o estudante configure o IDE *Spyder* de acordo com as instruções apresentadas na secção 3.2.

3.4 Integração da Placa de Som em *Python*

Em algumas UC será necessário usar a placa de som para reprodução e análise de sinais áudio. Para tal é necessário instalar a biblioteca *sounddevice* através do comando a ser executado na consola do *Spyder*:

```
conda install -c conda-forge python-sounddevice
```

procedimento este que poderá ser demorado. Após o processo de instalação terminar aparecerá a mensagem na consola do *Spyder* conforme apresentado na figura 3.16.



```

- conda-forge/label/spyder_rc
- conda-forge/label/spyder_dev
- conda-forge/label/spyder_kernels_rc
- conda-forge
Platform: win-64
Collecting package metadata (repodata.json): ...working... done
Solving environment: ...working... done

## Package Plan ##

  environment location: C:\Users\m_len\AppData\Local\spyder-6\envs\spyder-
runtime

```

Figura 3.16: Mensagens na consola após a instalação da biblioteca *sounddevice* via *conda*

Encontra-se disponibilizado na plataforma MOODLE o *script Introducao.py*, o qual pretende demonstrar as principais funcionalidades do *Python* que serão usadas na UC de Telecomunicações 1. Recomenda-se assim que só seja instalado esta biblioteca caso seja necessário usar sinais de áudio.

Na figura 3.17 encontra-se apresentado o ambiente *Spyder* após a execução do *script Introducao.py*, sendo apenas apresentado o valor da frequência de amostragem do sinal de fala usado neste exemplo, o qual deverá estar na mesma diretoria do *script*.

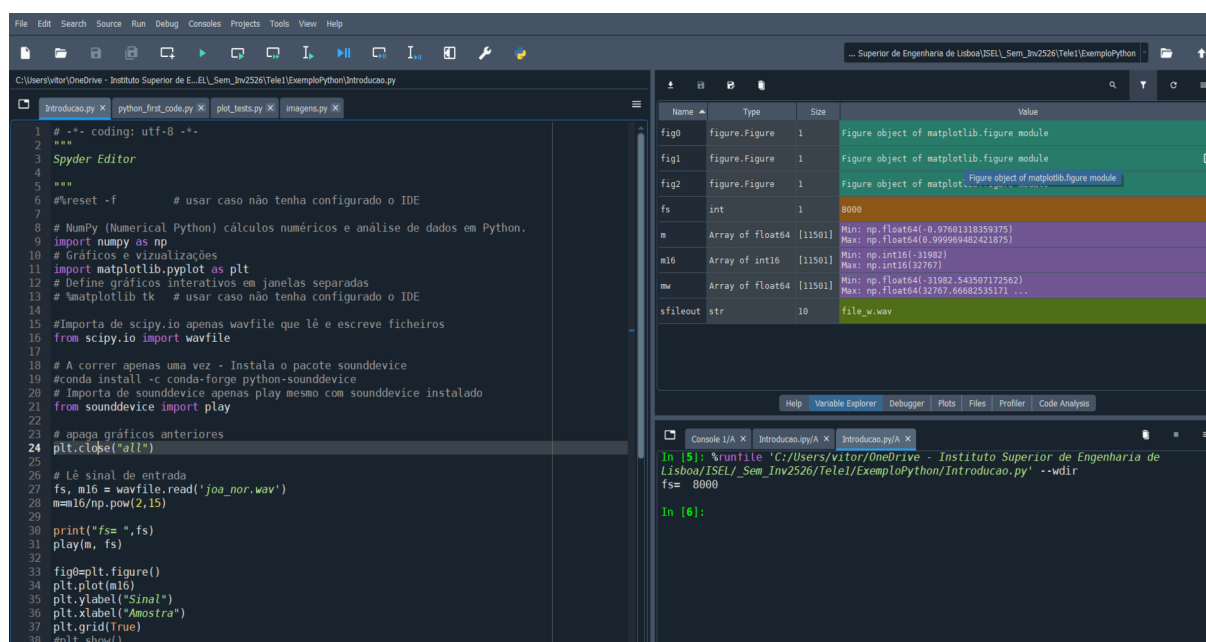


Figura 3.17: *Spyder* IDE após execução do *script Introducao.py*

Descrevem-se em seguida alguns comandos importantes que serão usados ao longo nos diversos trabalhos práticos.

O comando `%reset -f` apaga todas as variáveis em memória, incluindo os `import` das bibliotecas. Daí que este comando deverá estar na primeira linha do *script*. Caso já tenha procedido às configurações indicadas na secção 3.2, não é necessário usar esta linha de código.

O comando `import numpy as np` importa a biblioteca *numpy*, sendo que os métodos nela existentes serão instanciados como `np`.

O comando `import matplotlib.pyplot as plt` importa o módulo *pyplot* da biblioteca *matplotlib*, sendo que os métodos nela existentes serão instanciados como `plt`. O comando `%matplotlib tk` permite representar os gráficos criados através da instância `plt` em figuras isoladas e com permissões para gravá-las. Na secção 4.4 apresenta-se mais detalhes sobre a visualização de gráficos/sinais através da biblioteca *matplotlib*. Os gráficos obtidos neste *script* encontram-se na figura 3.18.

O comando `from scipy.io import wavfile` importa o módulo *wavfile* da biblioteca *SciPy*. O mesmo se passa para o comando `from sounddevice import play` importa o módulo *play* da biblioteca *sounddevice*.

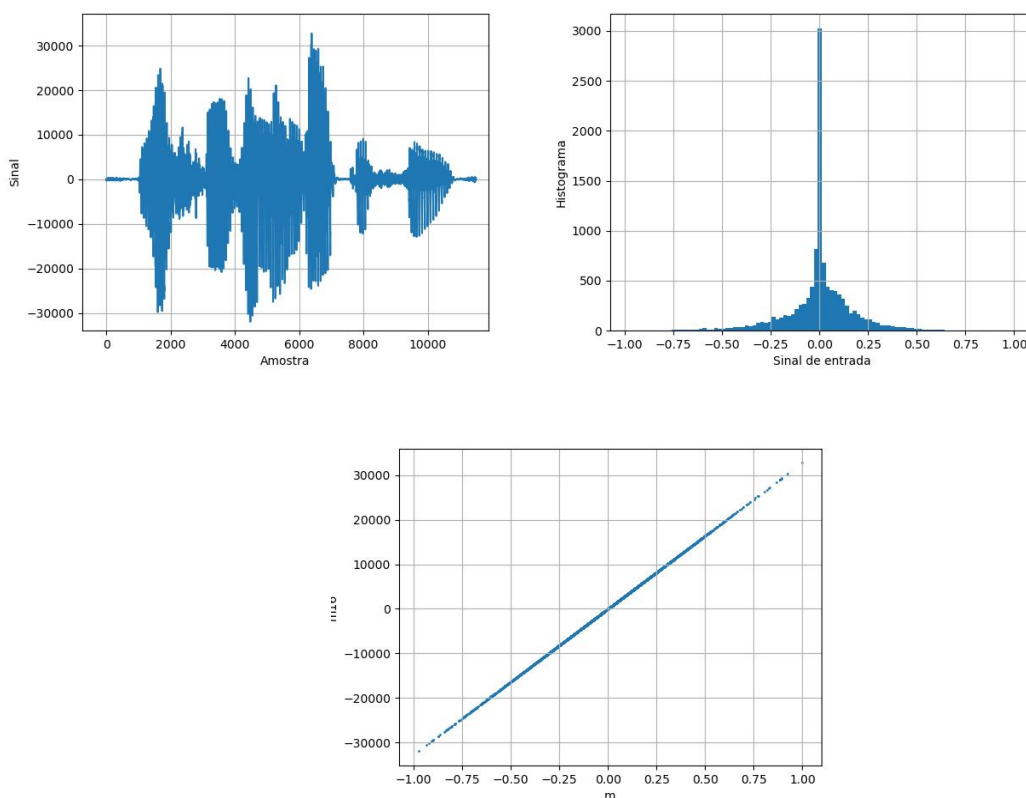


Figura 3.18: Resultado dos três gráficos do *script* *Introducao.py*

4 Introdução à Linguagem *Python*

Neste capítulo realiza-se uma breve introdução à linguagem *Python* com ênfase na análise e processamento de sinais.

O *Python* consolidou-se como uma das linguagens mais utilizadas no ensino e na investigação científica, destacando-se pela sua sintaxe simples, pela ampla disponibilidade de bibliotecas e pela facilidade com que permite converter conceitos teóricos em aplicações práticas. Estas características fazem do *Python* uma ferramenta particularmente adequada para utilizadores em fase inicial de aprendizagem, permitindo que o foco seja colocado na compreensão dos conceitos de Sinais e Sistemas, em vez de na complexidade associada à programação. Através da utilização de bibliotecas específicas, esta linguagem permite a geração, manipulação e análise de sinais, bem como a representação gráfica dos resultados de forma clara e intuitiva. Desta forma, conceitos como sinais contínuos e discretos, sistemas lineares, convolução e resposta em frequência podem ser explorados de forma experimental, complementando e reforçando a abordagem teórica tradicional.

O presente documento utiliza a linguagem de programação *Python*, apesar de não ter como objetivo o ensino da programação nessa linguagem. Os conhecimentos de programação necessários são adquiridos nas unidades curriculares próprias. Neste contexto, o *Python* é utilizado apenas como uma ferramenta de apoio à análise, simulação e representação dos conceitos abordados na unidade curricular de Sinais e Sistemas.

Na linguagem de programação *Python*, como outra qualquer, todas as variáveis e operações são tratadas de **forma discreta**. Deste modo, os sinais representados correspondem sempre a versões discretizadas dos sinais teóricos de tempo contínuo. Esta abordagem permite a análise e simulação computacional dos conceitos estudados em Sinais e Sistemas, mantendo a distinção entre os modelos contínuos e as suas representações em tempo discreto.

4.1 Tipos de Variáveis

Em *Python*, como noutra linguagem de programação, uma variável é um nome que referencia um valor armazenado em memória. Ao contrário de linguagens como C ou Java, não é necessário declarar explicitamente o tipo da variável. O tipo é atribuído automaticamente em função do valor associado à variável, podendo inclusive mudar ao longo da execução do programa. Esta flexibilidade simplifica a escrita de código. Nos exemplos seguintes ilustram-se diferentes tipos de variáveis. Em cada instrução é incluído um comentário (#) que indica o tipo de variável correspondente.

Nesta e nas secções seguintes são apresentados diversos exemplos de *Python* desde operações aritméticas simples, manipulação de *strings*, estruturas de dados e visualização de sinais. Antes de testar os exemplos abaixo, deverá importar a biblioteca *numpy* conforme se indica em seguida:

```
import numpy as np
```

Na tabela 4.1 apresentam-se um conjunto de exemplos com as variáveis mais usadas em *Python* no contexto de Sinais e Sistemas e Telecomunicações 1.

Tabela 4.1: Resumo das variáveis mais usadas em *Python*

# TIPOS BASICOS	
a = 5	# tipo int
b = 2.5	# tipo float
c = -3	# tipo int
d = 2.0	# tipo float
e = 0.5	# tipo float
f = 3 + 2j	# tipo complex
g = a > b	# tipo boolean
# TEXTO	
nome = "Sinais e Sistemas"	# tipo string
# BOOLEANOS	
flag = True	# tipo boolean
# COLECOES	
nums = [1, 2, 3]	# tipo list (mutavel)
coords = (10, 20)	# tipo tuple (imutavel)
letras = {"a", "b", "c"}	# tipo set
peessoa = {"nome": "Ana", "idade": 30}	# tipo dict
letras = {"a", "b", "c"}	# tipo set (sem duplicados)
n = np.array([0, 1, 2, 3])	# tipo array

4.1.1 Conversão de Tipos de Variáveis

Em *Python*, apesar da atribuição automática de tipos simplificar a programação, existem situações em que é necessário converter explicitamente o tipo de uma variável. Esta conversão é comum e evita resultados inesperados ou erros de execução.

A conversão de tipos é frequentemente necessária no contexto da realização de operações entre variáveis de tipos distintos, do controlo do tipo do resultado de operações matemáticas e do processamento de dados provenientes de diferentes fontes ou bibliotecas.

O *Python* disponibiliza funções para conversão explícita de tipos (*cast*), como `int()`, `float()` e `complex()`.

A conversão para inteiro deve ser usada com cuidado, uma vez que pode implicar perda de informação, como no caso da conversão de um número real para inteiro, conforme se exemplifica em seguida.

Tabela 4.2: Conversão para inteiro

```
x = int(3.7) # convercao para inteiro
```

Tabela 4.3: Conversão para *float*

```
a = 5  
b = float(a) # variavel inteira é convertida para float (numero real).
```

Em *Python*, um número real ou inteiro pode ser convertido explicitamente para um número complexo, conforme se exemplifica em seguida.

Tabela 4.4: Conversão para número complexo

```
a = 3  
z = complex(a)    # Variavel complexa com parte imaginaria nula  
  
x = 2.5  
z = complex(0, x) # Variavel complexa com parte real nula
```

Para verificar o tipo de variável usa-se o instrução `type()`, conforme apresentado em seguida.

Tabela 4.5: Verificação do tipo de variável

```
a = 3  
type(a)
```

4.2 Operações Básicas

O código seguinte declara duas variáveis distintas, uma do tipo inteiro e outra *float* e apresenta-as na consola através do comando `print()`.

Tabela 4.6: Exemplo com variáveis `int` e `float` usadas em *Python*

```
a = 3
b = -2.5
print(a)
print(b)
```

4.2.1 Operações Aritméticas Simples

A seguintes instruções permitem realizar somas algébricas, multiplicações e divisões. A variável `g` corresponde à potência de 2 (quadrado) do variável `b`.

Tabela 4.7: Resumo de algumas operações aritméticas simples usadas em *Python*

```
c=a+b    # soma
d=a-b    # subtracao
e=a/b    # divisao
f=a*b    # multiplicacao
g=b**2   # potenciacao ( potencia de 2)
print(c)
print(d)
print(e)
print(f)
print(g)
```

O operador `//`, designado por divisão inteira em *Python*, calcula o quociente da divisão entre dois operandos, devolvendo apenas a parte inteira do resultado. Esta operação corresponde a uma divisão arredondado por defeito, uma vez que o valor obtido é sempre arredondado para o menor inteiro igual ou inferior ao quociente da operação de divisão. Este operador é particularmente útil em situações que envolvem cálculos discretos, contadores, indexação e conversões de unidades.

Tabela 4.8: Exemplo do operador resto de divisão inteira

```
7 / 2     # 3.5    (divisão)
7 // 2    # 3      (divisao inteira por 2)
-7 / 2    # -3.5   (divisão)
-7 // 2   # -4     (divisao inteira por 2)
```


4.2.2 Operações Lógicas

As operações lógicas permitem realizar comparações e testes condicionais sobre variáveis. Em *Python*, estas operações produzem resultados do tipo booleano (*True* ou *False*) e são fundamentais na definição de sinais por intervalos e na implementação de sistemas. Cada uma das expressões apresentadas na tabela 4.9 devolve um valor lógico.

Tabela 4.9: Operadores de Comparação

```
a = 5
b = 3

print(a > b)    # a maior que b
print(a < b)    # a menor que b
print(a == b)   # a igual a b
print(a != b)   # a diferente de b
print(a >= b)   # a maior ou igual que b
print(a <= b)   # a menor ou igual que b
```

Na tabela 4.10 encontram-se alguns operadores lógicos que permitem combinar variáveis e/ou expressões booleanas.

Tabela 4.10: Operadores Lógicos

```
p = True
q = False

print(p and q)  # p E q
print(p or q)   # p OU q
print(not p)    # NAO p
```

Na tabela 4.11 encontra-se exemplificada a operação lógica XOR *bitwise*. Este operador, representado pelo símbolo $\wedge$ em *Python*, realiza uma operação lógica exclusiva bit a bit entre dois operandos inteiros. Para cada posição binária, o resultado é 1 apenas quando os bits comparados são diferentes; caso contrário, o resultado é 0. Esta operação é frequentemente usada em contextos como manipulação de bits, criptografia/cifra e técnicas de controlo de erros.

Tabela 4.11: Operador XOR *bitwise*

```
a = 5          # 0101
b = 3          # 0011

resultado = a ^ b # operador XOR bitwise
print(resultado) # 6 -> 0110
```

Por vezes é necessário definir intervalos temporais em sinais ou outro tipo de situações. Na tabela 4.12 apresentam-se alguns exemplos de operadores lógicos com variáveis numéricas.

Tabela 4.12: Operadores lógicos: exemplo com variáveis numéricas

```
n = 4

print(n >= 0)
print(n < 10)

n = 5
print((n >= 0) and (n <= 10))
```

Na tabela 4.13 apresenta-se um exemplo de aplicação a um conjunto de valores (*array*), no qual é criada uma condição lógica que permite identificar os elementos correspondentes a índices positivos. Nas secções 4.2.3 e 4.2.4 descrevem-se com mais detalhes operações usando *arrays*.

Tabela 4.13: Operadores lógicos: exemplo aplicado a um conjunto de valores (*array*)

```
import numpy as np

n = np.arange(-5, 6)
condicao = n >= 0
print(condicao)
```

4.2.3 Operações Aritméticas com Arrays

Na instrução seguinte, apresenta-se a criação de um *array* com quatro elementos designado por x , cuja representação analítica dada pela expressão (4.1) Para tal recorre-se ao método `array()` existente na biblioteca `numpy`. Este *array* é apresentado na consola através da instrução `print(x)`.

Tabela 4.14: Criação de um vetor uni dimensional (*array*)

```
x = np.array([10, 20, 30, 40])  
print(x)
```

$$x[n]=[10 \ 20 \ 30 \ 40] \quad (4.1)$$

É essencial ter conhecimento da forma de aceder ao conteúdo do *array*. Para tal apresentam-se na tabela 4.15 duas instruções que permitem aceder aos conteúdos da primeira e terceira posições do *array* x .

Tabela 4.15: Aceder ao conteúdo de um *array*

```
print(x[0]) # primeira amostra  
print(x[2]) # terceira amostra
```

Para determinar a quantidade de elementos de um determinado *array*, pode recorrer-se a uma das seguintes instruções: `len()` ou `size()`. Como se trata de um *array* uni-dimensional, neste exemplo o resultado é igual. O caso com *array* multi-dimensional o resultado está na secção 4.2.5.

Tabela 4.16: Aceder ao conteúdo de um *array*

```
N1 = len(x)  
N2 = np.size(x)  
print(N1)  
print(N2)
```

À semelhança das operações algébricas apresentadas na secção 4.2.1, é possível realizar as mesmas com *arrays*, conforme se apresenta em seguida.

No exemplo seguinte, apresentado na tabela 4.17, considera-se a soma de dois *arrays*, de acordo com a expressão (4.2).

$$z[n]=x[n]+y[n] \quad (4.2)$$

Tabela 4.17: Exemplo de soma entre dois *arrays*

```
x = np.array([10, 20, 30])
y = np.array([40, 50, 60])
z = x + y
print(z)
```

O exemplo apresentado na tabela 4.18 corresponde à subtração de dois *arrays*, de acordo com a expressão (4.3).

$$z[n] = x[n] - y[n] \quad (4.3)$$

Tabela 4.18: Exemplo de subtração entre dois *arrays*

```
x = np.array([10, 20, 30])
y = np.array([40, 50, 60])
z = x - y
print(z)
```

A expressão (4.4) define a multiplicação entre dois *arrays*, a qual apenas é válida quando ambos apresentam dimensões iguais, conforme apresentado na tabela 4.19 .

$$z[n] = x[n] \times y[n] \quad (4.4)$$

Tabela 4.19: Exemplo de multiplicação entre dois *arrays*

```
x = np.array([10, 20, 30])
y = np.array([40, 50, 60])
z = x * y
print(z)
```

A tabela 4.20 apresenta um exemplo de código que permite efetuar a potenciação de um *array* e a multiplicação por uma constante, conforme a expressão matemática (4.5).

$$y[n] = 2x^2[n] \quad (4.5)$$

Tabela 4.20: Exemplo de potenciação de um *array* com multiplicação por uma constante

```
x = np.array([10, 20, 30])
a = 2
y = 2*(x**2)
z = a*x
print(x)
print (y)
```

4.2.4 Instruções Especiais com *Arrays*

Nesta secção são apresentadas algumas instruções especiais para a manipulação de *arrays*, as quais são particularmente úteis no âmbito da disciplina de Sinais e Sistemas.

A função `arange()` pertence à NumPy e é usada para criar um *array* de valores numéricos sequenciais, definidos a partir de um valor inicial, um valor final e um passo fixo entre cada elemento. O intervalo começa no valor inicial e termina antes do valor final, que não é incluído no *array*. Na tabela 4.21 apresenta-se um exemplo de indexação lógica, onde o conjunto de valores n variam entre -5 a 6 com um passo de 1.

Tabela 4.21: Seleção de Elementos (Indexação Lógica): função `arange()`

```
n = np.arange(-5, 6)
x = n**2
y=x[n >= 0]
print(n)
print(x)
print(y)
```

Outra função *Python* usada para gerar sequências numéricas é o `linspace()`. Esta função pertence à biblioteca *NumPy* usada para gerar um conjunto de valores uniformemente espaçados entre dois limites. Ao contrário da função `arange()`, em que se define o passo entre os valores, a função `linspace()` permite definir quantos valores se são usados no intervalo definido. A função recebe um valor inicial, um valor final e o número de pontos a gerar. Por omissão, o valor final é incluído, o que torna a função `linspace()` especialmente útil no contexto de processamento de sinal. Esta função é muito usada na representação de funções contínuas, na criação de gráficos e em cálculos numéricos, uma vez que garante um espaçamento regular. Na tabela 4.22 encontra-se apresentado um exemplo de aplicação idêntico ao apresentado na tabela 4.21, mas usando a função `linspace()`.

Tabela 4.22: Seleção de Elementos (Indexação Lógica): função `linspace()`

```
n = np.linspace(0, 10, 5)
print(n)
x = n**2
print(x)
y=x[n >= 5]
print(n)
print(x)
print(y)
```

Na tabela 4.23 apresenta-se outra forma de gerar uma sequência numérica, desta feita com a função `zeros()` da biblioteca NumPy. Esta função e a função `ones()` são úteis no âmbito de processamento de sinais.

Tabela 4.23: Modificação de Elementos

```
x = np.zeros(10)
x[0] = 10
print(x)
x[x < 5] = -1
print(x)
```

Nas tabelas 4.24 e 4.25 estão exemplificados exemplos para a repetição de valores e concatenação de *arrays*, respetivamente.

Tabela 4.24: Repetição de Valores

```
x = np.array([1, 2, 3])
print(x)
y = np.repeat(x, 2)
print(y)
z = np.repeat(x, 3)
print(z)
```

Tabela 4.25: Concatenação de *Arrays*

```
x1 = np.array([1, 2, 3])
x2 = np.array([4, 5])
x = np.concatenate((x1, x2))
print(x)
```

Na tabela 4.26 encontra-se exemplificado o processo de inversão temporal, também designado por espelhamento, conforme descrito matematicamente pela expressão (4.6). Note-se que esta operação é a inversão da ordem dos elementos, não exatamente $x[-n]$ no sentido formal. Verifica-se assim que em contexto computacional, a inversão temporal corresponde à inversão da ordem dos elementos do vetor.

$$y[n] = x[-n] \quad (4.6)$$

Tabela 4.26: Inversão Temporal (espelhamento)

```
x = np.array([1, 2, 3, 4])
y = x[::-1]
print(y)
```

Na tabela 4.27 apresentam-se alguns exemplos de funções de cálculo com *arrays*.

Tabela 4.27: Exemplos de funções de cálculo com *arrays*

```
x = np.linspace(0,5,10)
print(np.sum(x))      # apresenta a soma de todos os elementos do array x
print(np.max(x))      # apresenta o valor maximo dos elementos do array x
print(np.min(x))      # apresenta o valor minimo dos elementos do array x
```

4.2.5 Instruções Especiais com Matrizes (*Arrays* multi-dimensionais)

Nesta secção apresentam-se alguns exemplos de criação e manipulação de matrizes (*arrays* multi-dimensionais) em *Python*.

No início da aprendizagem da programação em *Python*, é frequente ocorrer alguma confusão entre os conceitos de lista e *array*. Nos exemplos que se seguem, procura-se clarificar e ilustrar as principais diferenças entre estas duas estruturas de dados.

Considere os exemplos apresentados na tabela 4.28. Nela estão indicadas duas listas: a `lista_1`, cujos elementos são de natureza mista, e a `lista_2`, composta exclusivamente por valores numéricos.

Tabela 4.28: Exemplos de criação de listas

```
lista_1 = [1, 2.5, "abc", True]
lista_2 = [ [10, 20] , [30, 40] , [50,60] ]
```

A `lista_1`, contém elementos de tipos de dados diferentes: *int*, *float*, *string* e *boolean*. Esta heterogeneidade de tipos é uma das principais características das listas em *Python*, tornando-as estruturas de dados flexíveis e adequadas para armazenar informação diversa. A `lista_2`, contém unicamente elementos do tipo *int*.

Na tabela 4.29 exemplifica-se a conversão das listas apresentadas para o formato *array*. A conversão para o `array_1` resulta na transformação do tipo das variáveis, que passam a ser representadas como um *array* de cadeias de caracteres (*strings*). A conversão para o `array_2` origina um *array* com três linhas e duas colunas, constituído exclusivamente por valores inteiros. Ao contrário das listas, os *arrays* são estruturas homogéneas, isto é, todos os seus elementos pertencem ao mesmo tipo de dados, o que possibilita uma utilização mais eficiente da memória e um desempenho superior em operações matemáticas. Este tipo de estrutura revela-se particularmente adequado para cálculo numérico, manipulação matricial e aplicações de natureza científica.

Tabela 4.29: Exemplo de conversão de lista para *array*

```
array_1 = np.array(lista_1)
array_2 = np.array(lista_2)
```

Na tabela 4.30 encontra-se um exemplo de criação de duas matrizes *A* e *B* com inteiros. A instrução *A.shape* e *B.shape* permite determinar as dimensões das matrizes *A* e *B*, retornando um par de valores correspondente, respetivamente, ao número de linhas e ao número de colunas. No caso da instrução *np.size(A)*, esta devolve o número total de elementos da matriz.

Tabela 4.30: Criação de um *array* multi-dimensional (matriz)

```
A = np.array([[10, 20],[30, 40],[50,60]])      # cria o array A
B = np.array([[100, 200],[300, 400],[500,600]]) # cria o array B
Z = np.array([1, 2, 3]) # cria o array Z
print(A.shape)                                # retorna dimensao da matriz
print(B.shape)                                # retorna dimensao da matriz
print(Z.shape)                                # retorna dimensao da matriz
print(np.size(A))                             # retorna o numero total de elementos da matriz A
print(np.size(Z))                             # retorna o numero total de elementos da matriz Z
```

Na tabela 4.31 encontra-se um exemplo de várias operações com *arrays* multi-dimensionais. Chama-se a atenção para o facto de que as operações seguintes correspondem a cálculo numérico e não a cálculo matricial. Assim, o valor do *array* *E* não resulta da multiplicação matricial das matrizes *A* e *C*, mas sim da multiplicação elemento a elemento dos respetivos *arrays*.

Tabela 4.31: Exemplos de operações com *arrays* multi-dimensionais

```
C = A + B      # soma de dois arrays
D = A - C      # subtracao de dois arrays
E = A * C      # multiplicacao ponto dos elementos da matrizes
F = np.sqrt(B) # raiz quadrada de cada elemento do array B
G = np.sqrt(A) # raiz quadrada
H = np.exp(A)  # exponencial
I = np.log10(A) # logaritmo de base 10
```

Na tabela 4.32 apresenta-se algumas instruções que permitem realizar cálculo matricial. Chama-se a atenção para a instrução *A@B*, a qual permite realizar a multiplicação matricial entre as matrizes *A* e *B*. Apresentam-se também as operações de transposição de matrizes, calculo do determinante e a matriz inversa.

Tabela 4.32: Exemplos com operações de cálculo matricial

```

A = np.array([[10, 20],[30, 40],[50,60]])          # 3x2
B = np.array([[100, 200, 300],[400, 500, 600]])    # 2x3
J = A @ B                                           # multiplicacao matricial
K = np.dot(A, B)                                   # idêntico a linha anterior
L = A.T                                             # matriz transposta
A = np.array([[10, 20],[30, 40]])                  # 2x2
np.linalg.det(A)                                   # determinante da matriz A
np.linalg.inv(A)                                   # inversa da matriz A

```

Na tabela 4.33 apresentam-se exemplos de indexação matricial. No essencial, o processo é idêntico ao utilizado na manipulação de vetores (*arrays* uni-dimensionais), no entanto, é necessário ter especial cuidado na especificação dos índices correspondentes às linhas e às colunas.

Tabela 4.33: Exemplos de indexação com matrizes

```

A[0, 0]      # elemento da primeira linha e primeira coluna
A[1, 0]      # elemento da segunda linha e primeira coluna

A[0, :]      # todos os elemento da primeira linha
A[:, 1]      # todos os elemento da segunda coluna

```

Na tabela 4.34 apresentam-se algumas funções úteis que permitem efetuar operações de agregação sobre os elementos de um *array* de forma eficiente. Este conjunto de instruções é amplamente utilizado não só no contexto de Sinais e Sistemas, mas também em análise de dados, estatística computacional entre outras, devido à sua simplicidade sintática e intuitiva, bem como pelo elevado desempenho computacional.

Na tabela 4.35 apresentam-se exemplos de cálculo de alguns indicadores típicos tais como energia, potência média e valor médio. Importa recordar que, em computação, os sinais são sempre discretos. Assim, apresentam-se de seguida as expressões correspondentes às referidas grandezas, dadas por (4.7), (4.8) e (4.9), respetivamente.

$$E_x = \sum_{n=-\infty}^{+\infty} x^2[n] \quad (4.7)$$

$$P_x = \frac{1}{N} \sum_{n=0}^{N-1} x^2[n] \quad (4.8)$$

$$m_x = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \quad (4.9)$$

Tabela 4.34: Exemplos de cálculo de energia, potência média e valor médio

```

x=np.linspace(0,10,100) # vetor com cem elementos de 0 a 10
y=np.ones(100)          # vetor com cem elementos com valor 1

Ex=np.sum(x**2)          # calculo da energia do vetor x
Px=np.mean(x**2)         # calculo da potencia media do vetor x
mx=np.mean(x)            # calculo do valor medio do vetor x

Ey=np.sum(y**2)          # calculo da energia do vetor y
Py=np.mean(y**2)         # calculo da potencia media do vetor y
my=np.mean(y)            # calculo do valor medio do vetor y

```

4.3 Controlo de Fluxo

À semelhança de outras linguagens de programação, em *Python* é possível a realização de controlo de fluxo através da decisão binária (`if`), ciclo `for`, `while` e teste `match-case`.

Uma característica fundamental do *Python* é a utilização da indentação para definir blocos de código. Ao contrário de outras linguagens que recorrem a chavetas ou palavras-chave específicas, em *Python* a organização do código é feita através do espaçamento no início das linhas. Esta abordagem contribui para a legibilidade do código, mas exige atenção por parte do utilizador, uma vez que uma indentação incorreta pode conduzir a erros de execução.

4.3.1 Decisão Binária *if-else*

A estrutura de decisão `if-else` permite executar diferentes blocos de instruções consoante o resultado de uma condição lógica, cuja sintaxe é a seguinte:

```

if condicao:
    # instruções executadas se a condição for verdadeira
else:
    # instruções executadas se a condição for falsa

```

Na Tabela 4.35 apresenta-se um exemplo simples da utilização de uma decisão binária, baseada no teste à variável velocidade.

Tabela 4.35: Exemplos de decisão binária

```
velocidade = 50
if velocidade < 50:
    print( 'Seguro' )
elif velocidade < 120:
    print( 'Rapido' )
else:
    print( 'Excesso de velocidade' )
```

4.3.2 Ciclo *for*

A estrutura de repetição *for* permite executar um conjunto de instruções para cada elemento de uma sequência ou ao longo de um intervalo de valores previamente definido, segundo a seguinte sintaxe:

```
for elemento in sequencia:
    # instruções executadas para cada elemento
```

Na tabela 4.36 apresenta-se um exemplo do uso de um ciclo *for*.

Tabela 4.36: Exemplos de um ciclo *for*

```
for i in range(10):
    print(i)
```

4.3.3 Ciclo *while*

A estrutura de repetição *while* permite executar repetidamente um conjunto de instruções enquanto uma determinada condição lógica se mantiver verdadeira. Esta construção é utilizada quando o número de iterações não é conhecido à partida e depende da avaliação contínua de uma condição.

```
while condicao:
    # instruções executadas enquanto a condição for verdadeira
```

Na tabela 4.37 apresenta-se um exemplo do uso de um ciclo `while`.

Tabela 4.37: Exemplos de um ciclo `while`

```
i = 0
while i < 10:
    print( i )
    i += 1
```

4.3.4 Match-Case

Em *Python* não existe um *switch-case* tradicional como em C ou Java. Contudo, a partir do *Python* 3.10, existe a estrutura *match-case*, que desempenha função semelhante. Chama-se a atenção que o *match-case* funciona bem para valores exatos, mas não é adequado para intervalos numéricos. Na tabela 4.38 apresenta-se um exemplo de aplicação da estrutura *match-case*.

`match` expressao:

```
case padrao_1:
    bloco_1
case padrao_2:
    bloco_2
case _:
    bloco_default
```

Tabela 4.38: Exemplos de um ciclo `match-case`

```
match x:
    case 1:
        print("Um")
    case 2:
        print("Dois")
    case _:
        print("Outro valor")
```

4.4 Representação Gráfica

A representação gráfica constitui uma ferramenta fundamental na análise de sinais e sistemas. Embora a formulação matemática permita descrever rigorosamente um sinal, é através da sua visualização que se torna possível compreender intuitivamente propriedades como amplitude, periodicidade, simetria, entre outros.

Para a visualização de gráficos em *Python* é necessário importar o módulo *pyplot* da biblioteca *matplotlib*, que contém as funções utilizadas para criar gráficos em *Python*. Para tal recorre-se ao seguinte comando:

```
import matplotlib.pyplot as plt
```

Na tabela 4.39 apresenta-se o código para a geração e representação de um sinal sinusoidal dado pela expressão (4.10).

$$x(t)=3+2\cos(2\pi 100t) \quad (4.10)$$

Conforme se pode observar, numa primeira fase são importadas as bibliotecas e módulos necessários à execução do programa. Neste exemplo, utilizam-se a biblioteca *NumPy*, para a manipulação numérica e criação de *arrays*, e o módulo *pyplot* da biblioteca *Matplotlib*, responsável pela representação gráfica do sinal.

Após a definição das variáveis globais, surge a instrução:

```
t = np.linspace(0, dur - 1/fs, int(fs * dur))
```

esta instrução tem como objetivo criar um array cujos valores variam entre 0 e $\text{dur}-1/\text{fs}$, contendo um total de $\text{int}(\text{fs} * \text{dur})$ pontos igualmente espaçados.

O array *t* representa o vetor temporal amostrado do sinal. Nesta instrução, as variáveis têm o seguinte significado: *dur* corresponde à duração total do sinal; $1/\text{fs}$ representa o período de amostragem; *fs* é a frequência de amostragem e $\text{fs}*\text{dur}$ determina o número total de amostras no intervalo considerado.

A utilização de $\text{dur}-1/\text{fs}$ como limite superior garante que o último ponto corresponde à última amostra válida antes de ultrapassar a duração especificada. No exemplo considerado, sendo a frequência de amostragem igual a 8000 Hz e a duração correspondente a quatro períodos do sinal, obtêm-se 320 amostras no total.

A função `int()` é utilizada para assegurar que o número de pontos é um valor inteiro, evitando erros associados à criação do array.

Na tabela 4.40 apresenta-se o método de cálculo da energia, potência média e valor médio do sinal sinusoidal da expressão (4.10). O gráfico resultante encontra-se representado na figura 4.1.

Tabela 4.39: Exemplos da criação de uma figura com um sinal sinusoidal

```

import numpy as np
import matplotlib.pyplot as plt

A=2                # amplitude do co-seno
DC=3               # componente DC
f=100              # frequencia do co-seno
T=1/f              # período do co-seno
fase=0             # fase inicial
fs=8000            # frequência de amostragem
dur=4*T            # numero de periodos ( 4 )

# criacao do eixo dos 'tempos'
t=np.linspace(0,dur-1/fs,int(fs*dur)) # val. inicial; val. final; num. pontos
x=DC+A*np.cos(2*np.pi*f*t+fase)      # funcao co-seno

fig=plt.figure()                      # criação de um objeto figura
plt.title('Função co-seno')           # titulo da figura
plt.ylabel("x(t)",fontsize=12)         # label do eixo ordenadas
plt.xlabel("t [ms]",fontsize=12)       # label do eixo da abcissas
plt.plot(t/1e-3, x,linewidth=3)       # representacao do grafico
plt.grid(True)                        # grelha
plt.show()

```

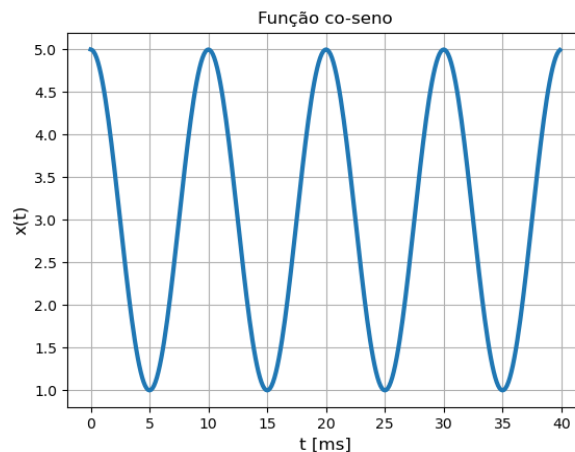
**Figura 4.1:** Representação gráfica do sinal sinusoidal

Tabela 4.40: Cálculo da energia, potência e valor médio do sinal sinusoidal

```
Ex=np.sum(x**2)
Px=np.mean(x**2)
mx=np.mean(x)
print(Ex)
print(Px)
print(mx)
```

4.5 Funções

Em *Python*, assim como em outras linguagens de programação, é possível definir funções como forma de organizar o código de maneira estruturada e hierarquizada. As funções permitem isolar blocos de instruções responsáveis por tarefas específicas, contribuindo para uma abordagem mais sistemática na resolução de problemas. Essa organização favorece a reutilização de procedimentos, reduz a redundância no código e facilita tanto a sua compreensão como a sua manutenção ao longo do tempo.

Para definir uma função em *Python* utiliza-se a palavra-chave `def`, seguida do nome da função e, entre parênteses, dos respectivos parâmetros de entrada. De forma geral, inclui-se uma breve descrição da finalidade da função, recorrendo a uma *docstring*, delimitada por aspas triplas (""") colocadas no início e no fim do texto descritivo. No final do bloco de instruções, os valores a devolver são especificados através da instrução `return`.

```
def nomeFuncao(inputargs):
    """
        breve descricao da funcao
    """
    #Codigo da funcao
    return outputargs1, outputargs2
```

Quando se trata de funções de utilização frequente, é prática recomendada defini-las num ficheiro separado, com extensão `.py`, conforme ilustrado na tabela 4.41, e posteriormente invocá-las no programa principal (*main*). Desta forma, promove-se uma organização mais clara do código e uma melhor reutilização dos diferentes componentes.

Através da instrução `import`, é possível carregar o módulo ou a biblioteca onde a função se encontra definida, tornando-a disponível no programa principal, tal como exemplificado na tabela 4.42.

A função `myParabola()`, apresentada na tabela 4.41, tem como argumentos de entrada os parâmetros `a`, `b` e `c`, além do vetor `t`, que representa as amostras no domínio do tempo. Com base nesses valores, a função calcula a expressão quadrática indicada em (4.11), determinando o valor do sinal em cada instante temporal (amostra) contida no *array* `t`.

$$y(t) = at^2 + bt + c \quad (4.11)$$

É possível consultar a descrição de uma função sem necessidade de abrir o ficheiro `.py` onde esta se encontra definida. Conforme indicado na tabela 4.42, a função `myParabola` é importada a partir do ficheiro `exemploFuncoes`, sendo utilizada no programa principal através do prefixo `ef`. Dessa forma, para aceder à documentação associada à função (*docstring*), basta executar na consola o seguinte comando:

```
help(ef.myParabola)
```

Tabela 4.41: Exemplo da função *myParabola* definida no ficheiro *exemploFuncoes.py*

```
import numpy as np
def myParabola(a,b,c,t):
    """
    Retorna as amostras de uma funcao quadratica (at^2+bt+c)
    Input:
        a,b,c e t
    Output:
        retorna as amostras
    """
    y=a*t**2 + b*t + c
    if a > 0:
        print("Parábola com concavidade para cima")
    elif a < 0:
        print("Parábola com concavidade para baixo")
    else:
        print("Função linear")
    return y
```


Na tabela 4.42 apresenta-se um exemplo de código que demonstra como representar, numa única figura, três gráficos distintos, utilizando cores diferentes e incluindo a respetiva legenda para cada um deles. Dessa forma, é possível comparar visualmente os vários sinais ou resultados obtidos, garantindo uma apresentação clara e devidamente identificada.

Este exemplo ilustra a possibilidade de invocar a mesma função múltiplas vezes, recorrendo a diferentes parâmetros de entrada. Deste modo, é possível analisar o comportamento do sistema ou do sinal para distintos conjuntos de valores, mantendo a mesma estrutura de cálculo e promovendo a reutilização eficiente do código. O resultado do exemplo apresentado encontra-se apresentado na figura 4.2.

Tabela 4.42: Chamada às funções apresentadas na tabela 4.41

```
import numpy as np
import exemploFuncoes as ef

t=np.linspace(-5,5,50)
a,b,c = 2,0,5                                # afetacao das variaveis
y=ef.myParabola(a, b, c, t)
a,b,c = -2,0,5
w=ef.myParabola(a, b, c, t)
a,b,c = 0,1,5
z=ef.myParabola(a, b, c, t)

fig=plt.figure()                              # criacao de um objeto figura

plt.title('Representação de  $at^2+bt+c$  ')    # titulo da figura
plt.ylabel("Amplitude",fontsize=12)           # label do eixo ordenadas
plt.xlabel("t [s]",fontsize=12)               # label do eixo da abcissas
plt.plot(t, y,linewidth=3, label='2t2 + 5')  # representacao do grafico
plt.plot(t, w,linewidth=3, label='-2t2 + 5')  # representacao do grafico
plt.plot(t, z,linewidth=3, label='t + 5')     # representacao do grafico

plt.legend()                                  # apresentacao da legenda
plt.grid(True)                                # grelha
plt.show()
```

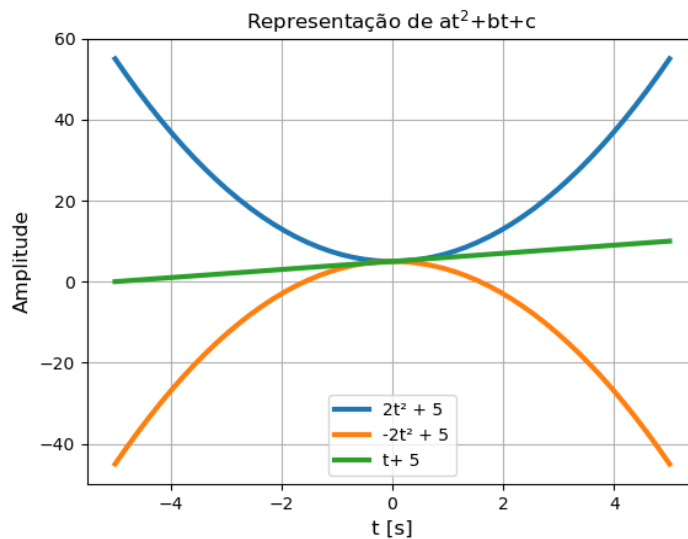


Figura 4.2: Representação dos sinais resultantes do exemplo apresentado na tabela 4.42

Na tabela 4.43 apresenta-se um exemplo de uma função que permite calcular o valor médio e o desvio padrão de um *array* fornecido como parâmetro de entrada. Este exemplo ilustra também a forma de devolver múltiplas variáveis através da instrução `return`. Importa salientar que algumas linhas de código não se encontram otimizadas, nomeadamente no cálculo do valor médio. Contudo, por se tratar de um documento de natureza didática, optou-se por apresentar uma implementação mais diretamente relacionada com a expressão teórica, privilegiando a clareza conceptual em detrimento da eficiência computacional.

Tabela 4.43: Função para cálculo da média e desvio padrão de um *array*

```
def statisticFunction(x):
    """
    Determina a media e o desvio padrao de um conjunto de valores x
    Input:
        x
    Output:
        media e desvio padrao
    """
    nelementos = np.size(x)
    media = np.sum(x)/nelementos      # media=np.mean(x)
    desvioPadrao=np.sqrt( (np.sum( x - media ))**2 / nelementos)
    return media,desvioPadrao
```

Na tabela 4.44 encontra-se um exemplo da geração de um sinal pseudo-aleatório com distribuição normal ou Gaussiana. Para tal tira-se partido da função `randn` da biblioteca `random` do *Python*. O código gera um vetor temporal `t` no intervalo de -1 a 1 com 8000 amostras e, com base nessa dimensão, é gerado um sinal de ruído branco gaussiano usando `np.random.randn`. O cálculo da média (*avg*) e o desvio padrão (*std*) do ruído são realizados pela função `statisticFunction` definida no módulo externo `exemploFuncoes`. É criada um figura com dois *subplots* verticais: no primeiro representa-se o sinal no domínio do tempo, e no segundo apresenta-se o histograma do ruído com 100 intervalos (classes).

Tabela 4.44: Exemplo de representação de dois gráficos distintos na mesma figura (subplot)

```
import numpy as np
import exemploFuncoes as ef

t=np.linspace(-1,1,8000)
ruído=np.random.randn(np.size(t));
[avg,std]=ef.statisticFunction(ruído)

fig, ax = plt.subplots(2, 1, figsize=(6, 6))

# Primeiro gráfico
ax[0].plot(t, ruído)
ax[0].set_title('Sinal no tempo')
ax[0].grid(True)

# Segundo gráfico
ax[1].hist(ruído,100)
ax[1].set_title('Histograma')
ax[1].grid(True)

plt.tight_layout()
plt.show()

print('Media:',avg)
print('Desvio Padrão:',std)
```

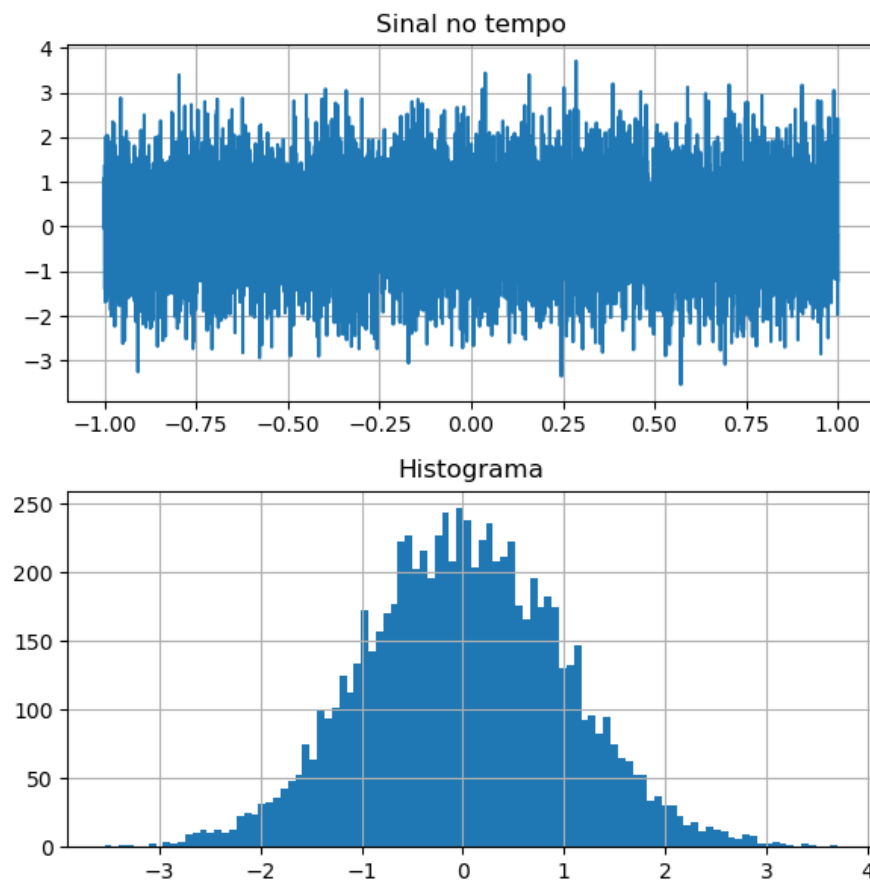


Figura 4.3: Resultado do código da tabela 4.44 - Sinal com ruído - tempo e histograma

4.6 Bibliotecas e Funções de Uso Corrente

Apresenta-se de seguida um conjunto de tabelas com as principais bibliotecas utilizadas em *Python* no âmbito da engenharia, com especial enfoque na área das telecomunicações/comunicações digitais. Para cada biblioteca são indicadas algumas das funções usadas mais frequentemente. Para cada uma é realizada uma breve descrição bem como a respetiva aplicação típica. A seleção apresentada não pretende ser exaustiva, mas sim fornecer uma referência prática e objetiva das ferramentas mais relevantes para o desenvolvimento de *scripts* de análise e processamento de sinais, modelação de sistemas e representação gráfica de resultados.

4.6.1 NumPy

Função	Descrição	Aplicação típica
<code>np.array()</code>	Criação de <i>arrays</i>	Definição de sinais discretos
<code>np.linspace()</code>	Geração de vetor temporal uniforme	Definição do eixo do tempo
<code>np.arange()</code>	Sequências com passo definido	Amostragem temporal
<code>np.zeros()</code>	<i>Array</i> de zeros	Inicialização de sinais
<code>np.ones()</code>	<i>Array</i> de uns	Sinais constantes
<code>np.mean()</code>	Valor médio	Análise estatística
<code>np.std()</code>	Desvio padrão	Caracterização de ruído
<code>np.sum()</code>	Soma de elementos	Energia do sinal
<code>np.dot()</code>	Produto interno	Operações vetoriais
<code>np.convolve()</code>	Convolução discreta	Sistemas LTI

4.6.2 *Matplotlib* (`matplotlib.pyplot`)

Função	Descrição	Aplicação típica
<code>plt.plot()</code>	Gráfico de linha	Representação de sinais
<code>plt.stem()</code>	Gráfico discreto	Sinais amostrados
<code>plt.subplot()</code>	Múltiplos gráficos	Comparação de sinais
<code>plt.figure()</code>	Nova figura	Organização gráfica
<code>plt.legend()</code>	Legenda	Identificação de sinais
<code>plt.title()</code>	Título	Contextualização
<code>plt.xlabel()</code> / <code>plt.ylabel()</code>	Designação dos eixos	Identificação de variáveis
<code>plt.grid()</code>	Grelha	Melhor leitura
<code>plt.hist()</code>	Histograma	Distribuição estatística

4.6.3 *SciPy*

Função	Descrição	Aplicação típica
<code>signal.convolve()</code>	Convolução	Resposta de sistemas
<code>signal.lfilter()</code>	Aplicação de filtro	Sistemas lineares
<code>signal.freqz()</code>	Resposta em frequência	Análise espectral
<code>signal.fftconvolve()</code>	Convolução via FFT	Processamento eficiente
<code>signal.step()</code>	Resposta ao degrau	Sistemas LTI
<code>signal.impulse()</code>	Resposta impulsional	Caracterização de sistemas
<code>signal.TransferFunction()</code>	Função de transferência	Modelação de sistemas

4.6.4 NumPy FFT (`np.fft`)

Função	Descrição	Aplicação típica
<code>np.fft.fft()</code>	Transformada de Fourier discreta	Análise espectral
<code>np.fft.ifft()</code>	Transformada inversa de Fourier	Reconstrução de sinal
<code>np.fft.fftfreq()</code>	Eixo de frequências	Representação espectral
<code>np.fft.fftshift()</code>	Espectro centrado em 0 Hz	Visualização

4.7 Links Úteis

Apresentam-se de seguida alguns recursos online que permitem aprofundar e consolidar os conhecimentos sobre a linguagem *Python*, constituindo um complemento útil ao estudo desenvolvido ao longo das várias unidades curriculares.

Sugere-se igualmente que sejam consultadas as ref

- Livro “[A Byte Of Python](https://python.swaroopch.com)” (<https://python.swaroopch.com>)
- Tutoriais sobre os conceitos e recursos básicos da linguagem e do sistema *Python*:
<https://docs.python.org/3/tutorial/index.html>
<https://www.w3schools.com/python/default.asp>
- Operações com números, strings e listas. Comandos na consola (prompt) do *Python*
<https://docs.python.org/3/tutorial/introduction.html#using-python-as-a-calculator>
- Expressões de teste e controlo de fluxo (`if`, `for` e `while`)
<https://docs.python.org/3/tutorial/controlflow.html>
- Operadores em *Python*
https://www.w3schools.com/python/python_operators.asp

Bibliografia

- [1] S. Swaroop, “A Byte of Python”, 2023. [Online]. Available: <https://python.swaroopch.com>
- [2] Python Software Foundation, “Python Documentation,” 2024. [Online]. Available: <https://docs.python.org/3/>
- [3] NumPy Developers, “NumPy Reference Guide,” 2024. [Online]. Available: <https://numpy.org/doc/>
- [4] A. V. Oppenheim, A. S. Willsky, and S. H. Nawab, *Signals and Systems*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1997.